Lecture 7: Markov Chain Monte Carlo

Professor Alexander Franks

2025-11-05

Reminders

- Quiz 3 today / normal, MCMC
- HW 3 due, HW 4 out tonight

$$y_1, \dots y_n \overset{iid}{\sim} N(\mu, \sigma^2)$$

known

$$\mu \sim N\left(\mu_0, \frac{\sigma^2}{k_0}\right)$$

$$P(\mu \mid y_1, \dots y_n) \sim N\left(w\bar{y} + (1-w)\mu_0, \frac{\sigma^2}{n+k_0}\right)$$

$$w = \frac{n}{n+k_0}$$

Bias - Var. Tradeoff

$w\bar{Y} + (1-w)\mu_0$

(random)

σ^2 also unknown

$$P(\mu, \sigma^2 \mid y_1, \dots y_n) \propto$$

$$L(\mu, \sigma^2)\, P(\mu \mid \sigma^2)\, P(\sigma^2)$$

$\propto 1/\sigma^2$

$N\left(\mu_0, \frac{\sigma^2}{k_0}\right)$

Monte Carlo

Monte Carlo estimation

LLN

- $\bar{\theta} = \sum_{s=1}^{S} \theta^{(s)}/S \rightarrow \mathrm{E}[\theta|y_1, \ldots, y_n]$
- $\sum_{s=1}^{S} \left(\theta^{(s)} - \bar{\theta}\right)^2/(S-1) \rightarrow \mathrm{Var}[\theta|y_1, \ldots, y_n]$
- $\#\left(\theta^{(s)} \leq c\right)/S \rightarrow \Pr(\theta \leq c|y_1, \ldots, y_n)$
- the α-percentile of $\left\{\theta^{(1)}, \ldots, \theta^{(S)}\right\} \rightarrow \theta_\alpha$

$L(\theta)P(\theta)$

Can write down posterior, but we want summaries (integrals)

Sampling from the posterior distributions

- The Monte Carlo methods we discussed previously assumed we could easily get samples from the posterior, e.g. with `rnorm`

- In general, sampling from a general probability distribution is hard

- Want to call `rcomplicatedistribution()` but don't have it

  - Inversion sampling is limited , Rejection sampling.
  - ~~Grid sampling is reasonable in 1 or 2 dimensions~~

- In high dimensions, these approaches aren't sufficient

M C M C

Markov Chain Monte Carlo

- We want independent random samples, $\theta^{(s)}$ from $p(\theta \mid y_1, \ldots y_n)$

Too Hard

- But there is no good way to get independent samples
- Alternative, create a sequence of **correlated** samples that converge to the correct distribution

we'll pay a price.

- Markov Chain Monte Carlo gives us a way to generate correlated samples from a distribution

Converge to the posterior.

This is the price we pay.

Monte Carlo Error

$Var(\sum_{i=1}^{n} Y_i) = \sum_{i,j} Cov(Y_i, Y_j)$ iff indep. $= \sum Var(Y_i)$

- Reminder: $\bar{\theta} = \sum_{s=1}^{S} \theta^{(s)} / S$ and S is the number of samples.
- If the samples are independent,

M.C. Error

$$\text{Var}(\bar{\theta}) = \frac{1}{S^2} \sum_{s=1}^{S} \text{Var}(\theta^{(s)}) = \frac{\text{Var}(\theta \mid y_1, \ldots y_n)}{S}$$

- If the samples are *positively correlated*, (as with MCMC)

$$\text{Var}(\bar{\theta}) = \frac{1}{S^2} \sum_{s,t} \text{Cov}(\theta^{(s)}, \theta^{(t)}) > \frac{\text{Var}(\theta \mid y_1, \ldots y_n)}{S}$$

- MCMC methods have higher Monte Carlo error due to positive dependence between samples.
- Hope to minimize dependence and thus MC error

MCMC error higher than MC

because of the correlation.

Basics of Markov Chains

Markov Chains: Big Picture

LLN

- For standard Monte Carlo, we make use of the law of large number to approximate posterior quantities
- The law of large numbers can still apply to random variables that are not independent
- We have a sequence of random variables indexed in "time", θ_t
- We'll be using a *discrete-time* Markov Chain: $t \in 0, 1, \ldots T$
- The observations, $\theta^{(t)}$ can be discrete or continous ("discrete-state" or "continuous-state" Markov Chain)

Discrete-state Markov Chains

- Let $\theta^{(t)} \in 1, 2, \ldots M$ be the state space for the Markov Chain

- A sequence is called a markov chain if

$$Pr(\theta^{(t+1)} \mid \theta^{(t)}, \cancel{\theta^{(t-1)} \ldots \theta^{(1)}}) = Pr(\theta^{(t+1)} \mid \theta^{(t)})$$

  for all $t \geq 0$

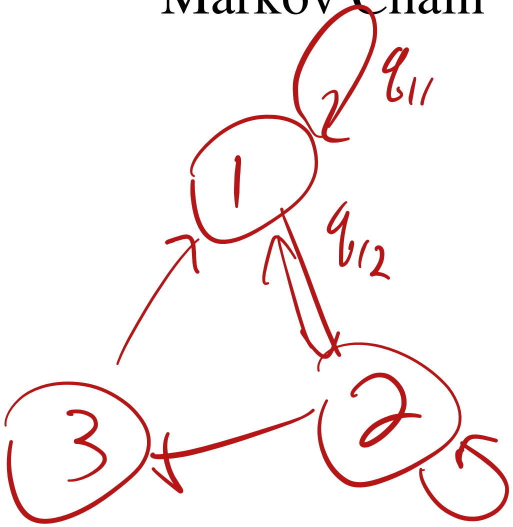
- The **Markov property**: given the entire past history, $\theta^{(1)}, \ldots \theta^{(t)}$, the most recent $\theta^{(t+1)}$ depends only on the immediate past, $\theta^{(t)}$

The Transition Matrix

- Define $q_{ij} = Pr(\theta^{(t+1)} \mid \theta^{(t)})$ is the transition probability from state i to state j

  TO From

- The $M \times M$ matrix $Q = (q_{ij})$ is called the *transition matrix* of the Markov Chain



3-state example

$$\begin{array}{cc} & \text{TO} \\ \text{From} \quad Q = & \begin{bmatrix} q_{11} & q_{12} & q_{13} \\ q_{21} & q_{22} & q_{23} \\ q_{31} & q_{32} & q_{33} \end{bmatrix} \end{array}$$

The Transition Matrix

3-state example

$$Q = \begin{bmatrix} q_{11} & q_{12} & q_{13} \\ q_{21} & q_{22} & q_{23} \\ q_{31} & q_{32} & q_{33} \end{bmatrix}$$

- The rows of the transition matrix sum to 1
- Note: $Q^n = (q_{ij}^{(n)})$ is the probability of transitioning from i to j in n steps

$$\underbrace{Q \times Q \times \cdots \times Q}_{n}$$

The limiting distribution

- A regular, irreducible Markov chain has a **limiting probability distribution**
  - Cover definitions of regular and irreducible in PSTAT160 (or related)
- Limit distribution describes the long-run fraction of time the Markov Chain spends in each state
  - *Does not* depend on where the chain starts
- Let $\pi = (\pi_1, \ldots \pi_M)$ be a row vector of probabilities associated with each state, such that $\sum_{i=1}^{M} = \pi_i = 1$
  - The limiting distribution converges to π, which is said to be **stationary** because $\pi Q = \pi$
  - If you sample from the limiting distribution and then transition, the result is still distributed according to the limiting distribution

Markov Chain Example

- Sociologists often study social mobility using a Markov chain.
- In this example, the state space is `{low income, middle income, and high income}` of families
- Let $\mathbf{Q}$ be the transition matrix from parents income to childrens income

Kids

$$\mathbf{Q} = \begin{array}{l|lll|} & \text{Lower} & \text{Middle} & \text{Upper} \\ \hline \text{Lower} & 0.40 & 0.50 & 0.10 \\ \text{Middle} & 0.05 & 0.70 & 0.25 \\ \text{Upper} & 0.05 & 0.50 & 0.45 \end{array}$$

Parents

Multi-step Transition Probabilities

2-step transition probabilities Grand kids

Parents

$$\mathbf{Q}^2 = \underline{\mathbf{Q} \times \mathbf{Q}} = \begin{vmatrix} 0.1900 & 0.6000 & 0.2100 \\ 0.0675 & \underline{0.6400} & \underline{0.2925} \\ 0.0675 & 0.6000 & 0.3325 \end{vmatrix}$$

4-step transition probabilities

$$\mathbf{Q}^4 = \mathbf{Q}^2 \times \mathbf{Q}^2 = \begin{vmatrix} 0.0908 & 0.6240 & 0.2852 \\ 0.0758 & 0.6256 & 0.2986 \\ 0.0758 & 0.6240 & 0.3002 \end{vmatrix}$$

Multi-step Transition Probabilities

4-step transition probabilities

$$\mathbf{Q}^4 = \mathbf{Q}^2 \times \mathbf{Q}^2 = \begin{vmatrix} 0.0908 & 0.6240 & 0.2852 \\ 0.0758 & 0.6256 & 0.2986 \\ 0.0758 & 0.6240 & 0.3002 \end{vmatrix}$$

8-step transition probabilities

$$\mathbf{Q}^8 = \mathbf{Q}^4 \times \mathbf{Q}^4 = \begin{vmatrix} 0.0772 & 0.6250 & 0.2978 \\ 0.0769 & 0.6250 & 0.2981 \\ 0.0769 & 0.6250 & 0.2981 \end{vmatrix}$$

Stationary distribution:

$$\pi = (\pi_1, \pi_2, \pi_3)$$

$$\pi_1 + \pi_2 + \pi_3 = 1$$

$$\begin{pmatrix} \pi_1 & \pi_2 & \pi_3 \\ \pi_1 & \pi_2 & \pi_3 \\ \pi_1 & \pi_2 & \pi_3 \end{pmatrix} = \lim_{n \to \infty} Q^n$$

The limiting distribution

$$\mathbf{Q}^{\infty} = \mathbf{1}\pi = \begin{vmatrix} \pi_1 & \pi_2 & \pi_3 \\ \pi_1 & \pi_2 & \pi_3 \\ \pi_1 & \pi_2 & \pi_3 \end{vmatrix}$$

```
Q <- matrix(c(0.4, 0.05, 0.05,
              0.5, 0.7, 0.5,
              0.1, 0.25, 0.45),
            ncol=3)

p <- eigen(t(Q))$vectors[, 1]
stationary_probs <- p/sum(p)
stationary_probs
```

π is related to eigenvectors of Q.

π

```
## [1] 0.07692308 0.62500000 0.29807692
```

```
stationary_probs %*% Q
```

```
##            [,1]  [,2]      [,3]
## [1,] 0.07692308 0.625 0.2980769
```

Get π back.

Markov Chain Monte Carlo

- Incredible idea: create a Markov Chain with the desired limiting distribution
  - Want the limiting distribution to be the posterior distribution
- Unlike the previous examples, we will mostly work with *infinite* state space

Transition Rule

- Want $\boxed{p(\theta^{(t+1)} \mid \theta^{(t)})}$ to have limiting distribution $p(\theta \mid y)$
  - If we run the random walk for long enough, $\theta^{(t)}$ will be distributed approximately according to $p(\theta \mid y)$

Analog of Q, transition matrix, for continuous states.

"Transition kernel."

Generalizing the Rejection Sampler.

- Sample from proposal $q(\theta)$
  - Doesn't need to envelope $p(\theta|y)$!
  - If $p(\theta|y) > 0$, then $q(\theta) > 0$

- Unlike the rejection sampler, we never "throw out" samples.

- At each iteration, choice:
  - Accept the proposed sample
  - <u>Or</u> <u>Repeat</u> the last sample.

The Independence Sampler

- The Metropolis algorithm tells us how to construct a transition matrix with the correct limiting distribution
    - The Independence Sampler is a special case of the Metropolis algorithm
- Sample from a proposal, $q(\theta)$. Best if $q(\theta)$ is close to $p(\theta \mid y)$.
- If $p(\theta \mid y) > 0$ then we need $q(\theta) > 0$
- At each iteration we have a choice:
    - Accept the new proposed sample
    - Or keep the previous sample for another iteration

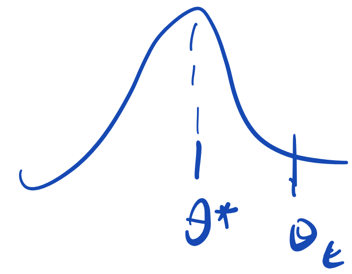
The Independence Sampler (modification of R.S.

1. Initialize θ_0 to be the starting point for you Markov Chain
2. Choose a *proposal distribution*, $J(\theta^*)$ (same as q before)
   - Propose a candidate value for the next sample
   - Best performance if density is very similar to target
3. Generate the candidate θ^* from the proposal distribution, J
4. Compute $r = \min(1, \frac{p(\theta^*|y)}{p(\theta_t|y)})$ "ratio of posterior evaluated at proposed div. by current"
5. Set $\theta_{t+1} \leftarrow \theta^*$ with probability r
   - Generate a uniform random number $u \sim Unif(0, 1)$
   - If $u < r$ we accept θ^* as our next sample
   - ~~Else $\theta_{t+1} \leftarrow \theta_t$ (we do not update the sample this time)~~

Else set $\theta_{t+1} \leftarrow \theta_t$

Intuition

$$r = \min\left(1, \frac{p(\theta^* | y)}{p(\theta_t | y)}\right)$$



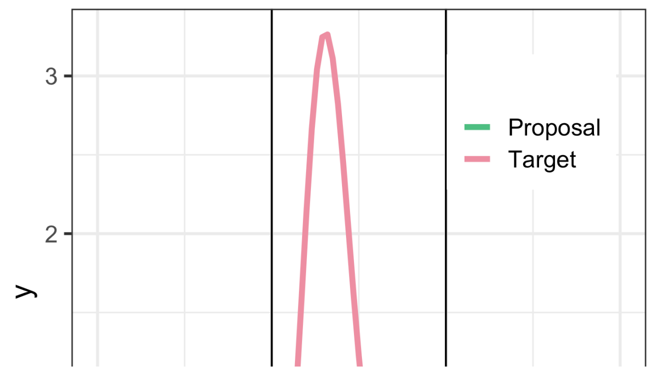
- If $p(\theta^* \mid y) > p(\theta_t \mid y)$ accept with probability 1
  - The proposed sample has higher posterior density than the previous sample
  - Always accept if we increase the posterior probability density
- If $p(\theta^* \mid y) < p(\theta_t \mid y)$ accept with probability $r < 1$
  - Accept with probability less than 1 if probability density would decrease
  - Relative frequency of θ^* vs θ_t in our samples should be $\frac{p(\theta^* | y)}{p(\theta_t | y)}$

An Example

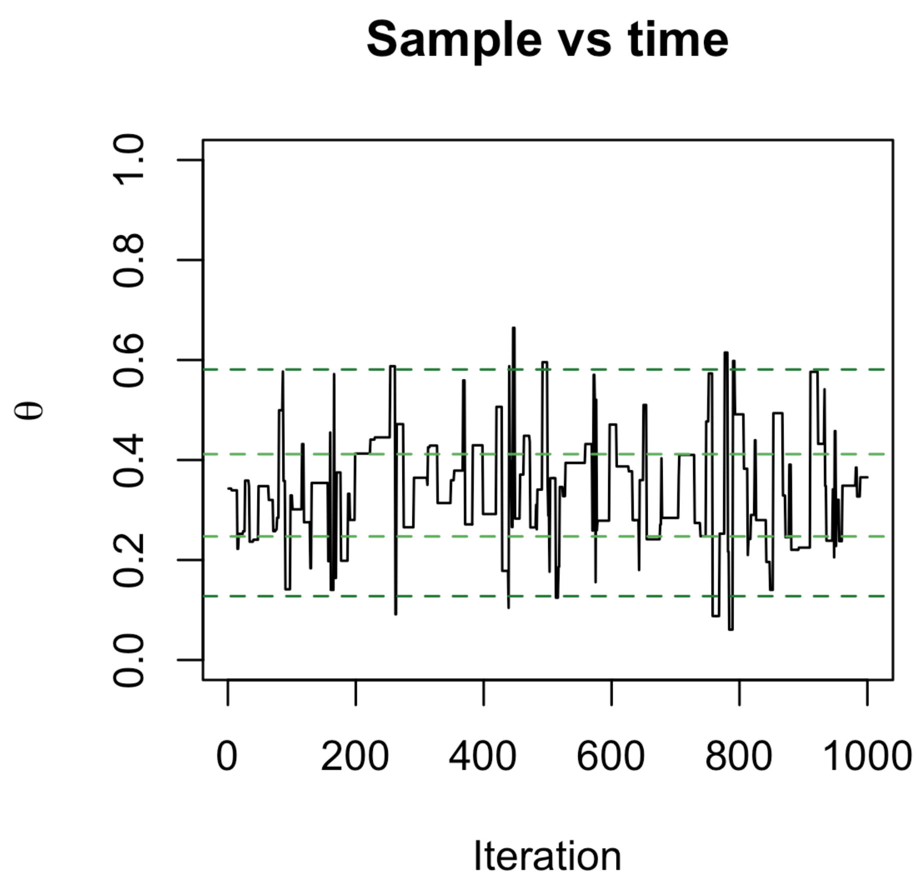
- Let $P(\theta \mid y)$ be a Beta$(5, 10)$ posterior distribution
- Propose from a distribution $J(\theta^*) \sim N(0.5, 1)$

```
## Warning: Using `size` aesthetic for lines was deprecated in ggplot2 3.4.
## ℹ Please use `linewidth` instead.
## This warning is displayed once every 8 hours.
## Call `lifecycle::last_lifecycle_warnings()` to see where this warning wa
## generated.

## Warning: A numeric `legend.position` argument in `theme()` was deprecate
## 3.5.0.
## ℹ Please use the `legend.position.inside` argument of `theme()` instead.
## This warning is displayed once every 8 hours.
## Call `lifecycle::last_lifecycle_warnings()` to see where this warning wa
## generated.
```

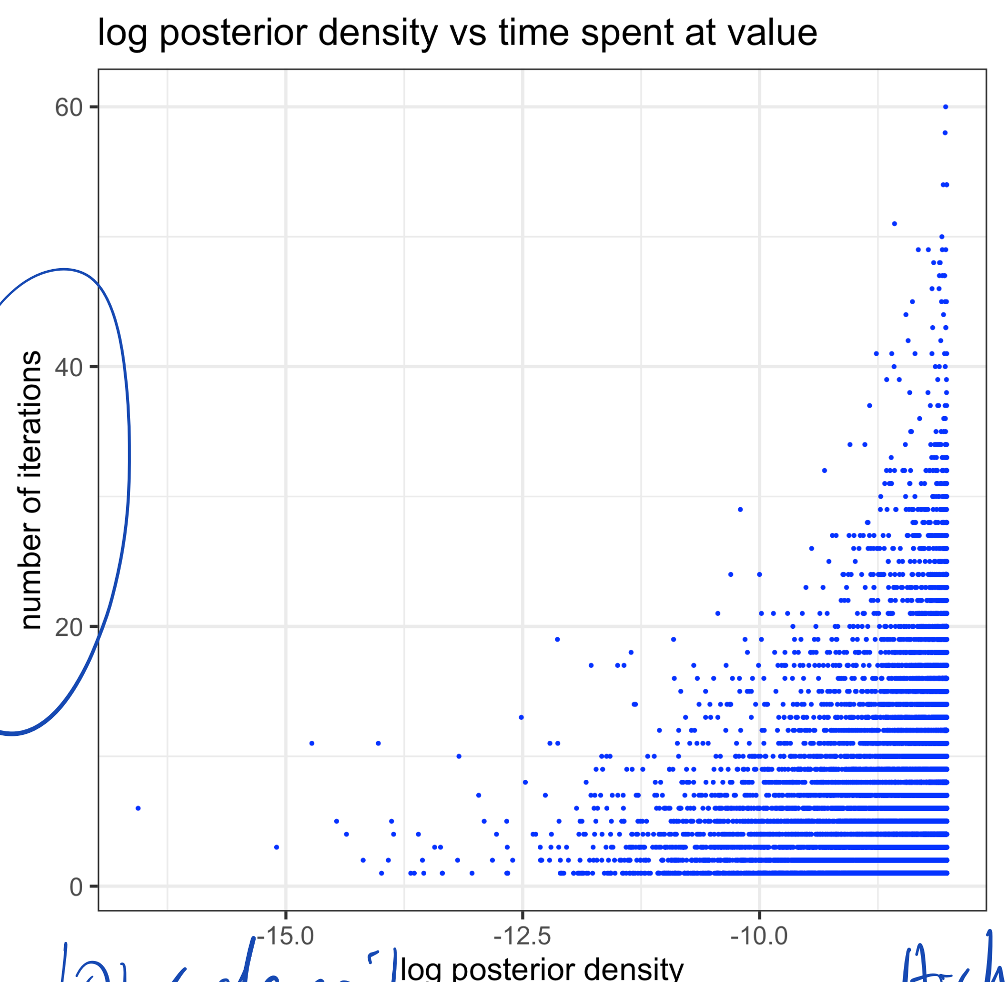


Independence Sampler



Note and source of confusion: samples are correlated over time for the "independence sampler".

Weighting by waiting

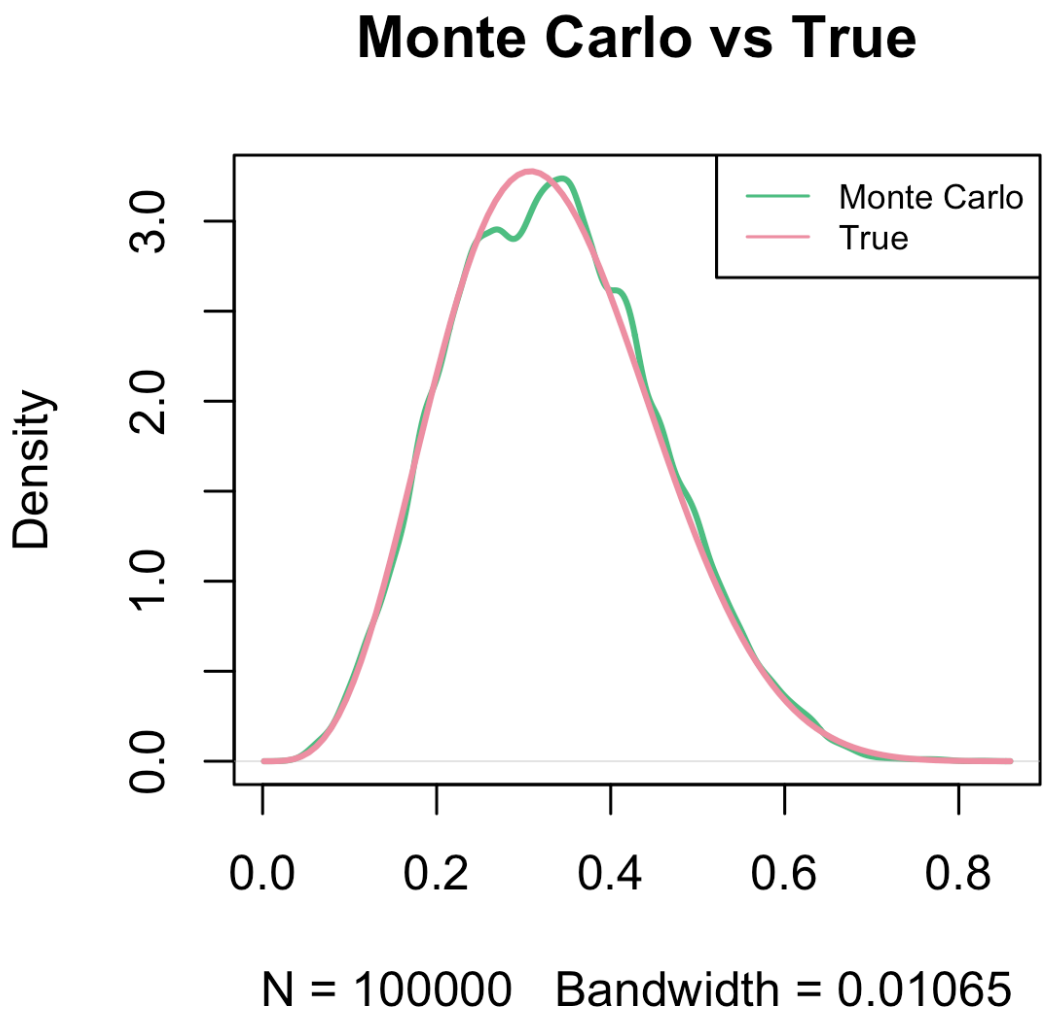


low density θ's

High density θ's

Where did the sampler get stuck? Where does it quickly leave?

Independence Sampler



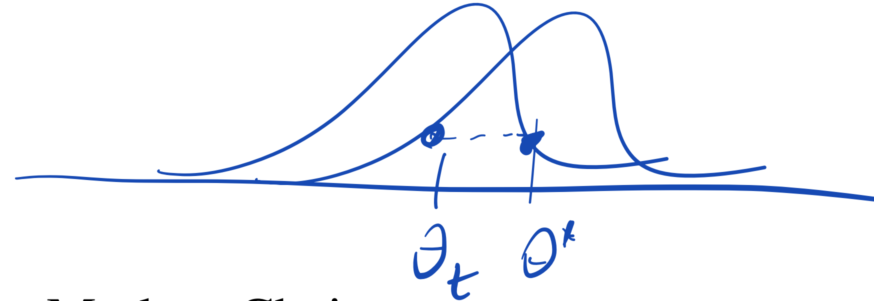
The Metropolis Algorithm

- Generalize the previous special case
- Allow the proposal distribution to depend on the most recent sample
    - Sometimes called an "Independence sampler": $J(\theta^*)$, e.g. $\theta^* \sim N(0.5, 1)$
    - Metropolis: $J(\theta^* \mid \theta_t)$, e.g. $\theta^* \sim N(\theta_t, 1)$

Propose new parameter given current

- Independence sampler: "Independence" refers to the proposal being fixed (the samples are **not** independent)!
- Metropolis sampler: a "moving" proposal distribution

The Metropolis Algorithm



1. Initialize θ_0 to be the starting point for you Markov Chain
2. Choose a proposal distribution, $J(\theta^* \mid \theta_t)$, e.g. $\theta^* \sim N(\theta_t, 1)$
   - Propose a candidate value for the next sample
   - Must have symmetry: $J(\theta^* \mid \theta_t) = J(\theta_t \mid \theta^*)$

   Equally likely to go from $\theta_t \to \theta^*$ as $\theta^* \to \theta_t$
3. Generate the candidate θ^* from the proposal distribution, J
4. Compute $r = \min(1, \frac{p(\theta^*|y)}{p(\theta_t|y)})$ same rule
5. Set $\theta_{t+1} \leftarrow \theta^*$ with probability r
   - Generate a uniform random number $u \sim Unif(0, 1)$
   - If $u < r$ we accept θ^* as our next sample
   - ~~Else $\theta_{t+1} \leftarrow \theta_t$ (we do not update the sample this time)~~

Else keep previous, $\theta_{t+1} \leftarrow \theta_t$

Aside: Arianna Rosenbluth

https://en.wikipedia.org/wiki/Arianna_W._Rosenbluth

Metropolis Algorithm

- Let $P(\theta \mid y)$ be a Beta(5, 10) posterior distribution
- 1-d sampling: lets try sampling from the Beta using the Metropolis algorithm
- Initialize θ_0 to 0.9
  - Note that the probability of drawing a value larger than 0.9 from a Beta(5, 10) is smaller than 1e-8
  - Our initial value is far from the high posterior density
  - In the long run this won't matter

How do I choose J?

- Define transition kernel $J(\theta_{t+1} \mid \theta_t)$ as $\theta^* \sim N(\theta_t, \tau^2)$
  - How does choice of τ^2 effect performance of MC sampler?

τ^2: Not too big,
not too small,
Goldilocks problem.

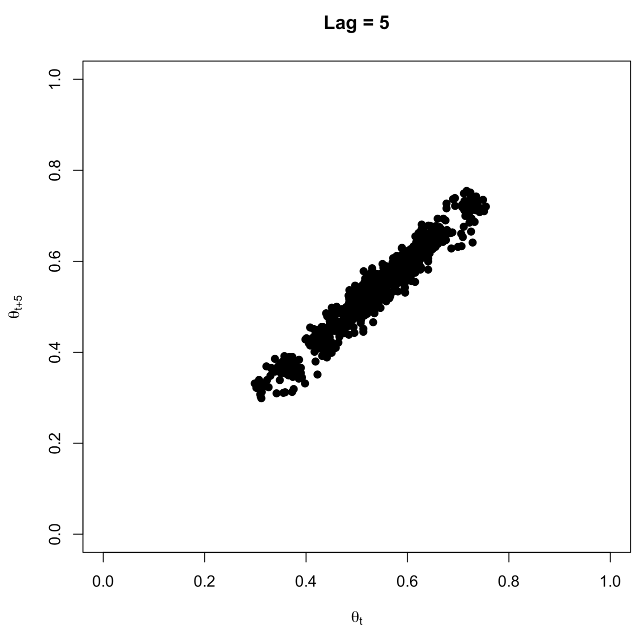
Demo

Autocorrelation of the Markov Chain

$\tau^2 = 0.01$ ("small" proposal variance)

Plot θ^t vs θ^{t+5} for all values of t

θ_{t+5}



θ_t

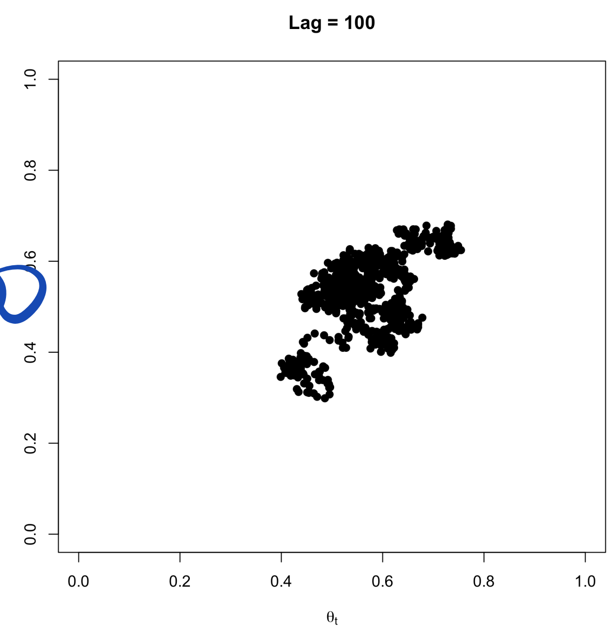
Correlation = 0.9643696

Autocorrelation of the Markov Chain

$\tau^2 = 0.01$ ("small" jump)

Plot θ^t vs θ^{t+100} for all values of t

θ_{t+100}



θ_t

Correlation = 0.382004

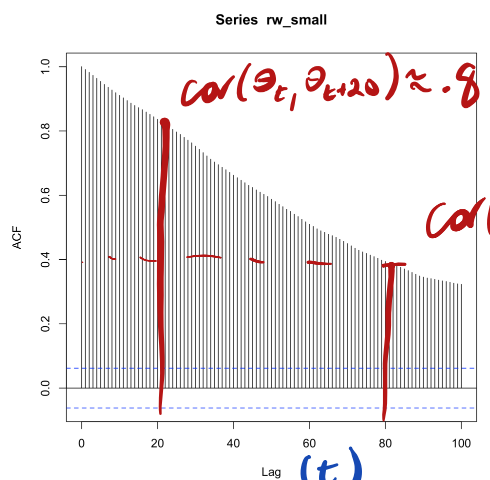
The Metropolis Algorithm

Auto-correlation Function (acf)

$\tau^2 = 0.01$ ("small" jump)

```
acf(rw_small, lag=100)
```

cor



(t)

$cor(\theta_t, \theta_{t+20}) \approx .8$

$cor(\theta_t, \theta_{t+80}) \approx .4$

want correlations to be small! (independence)

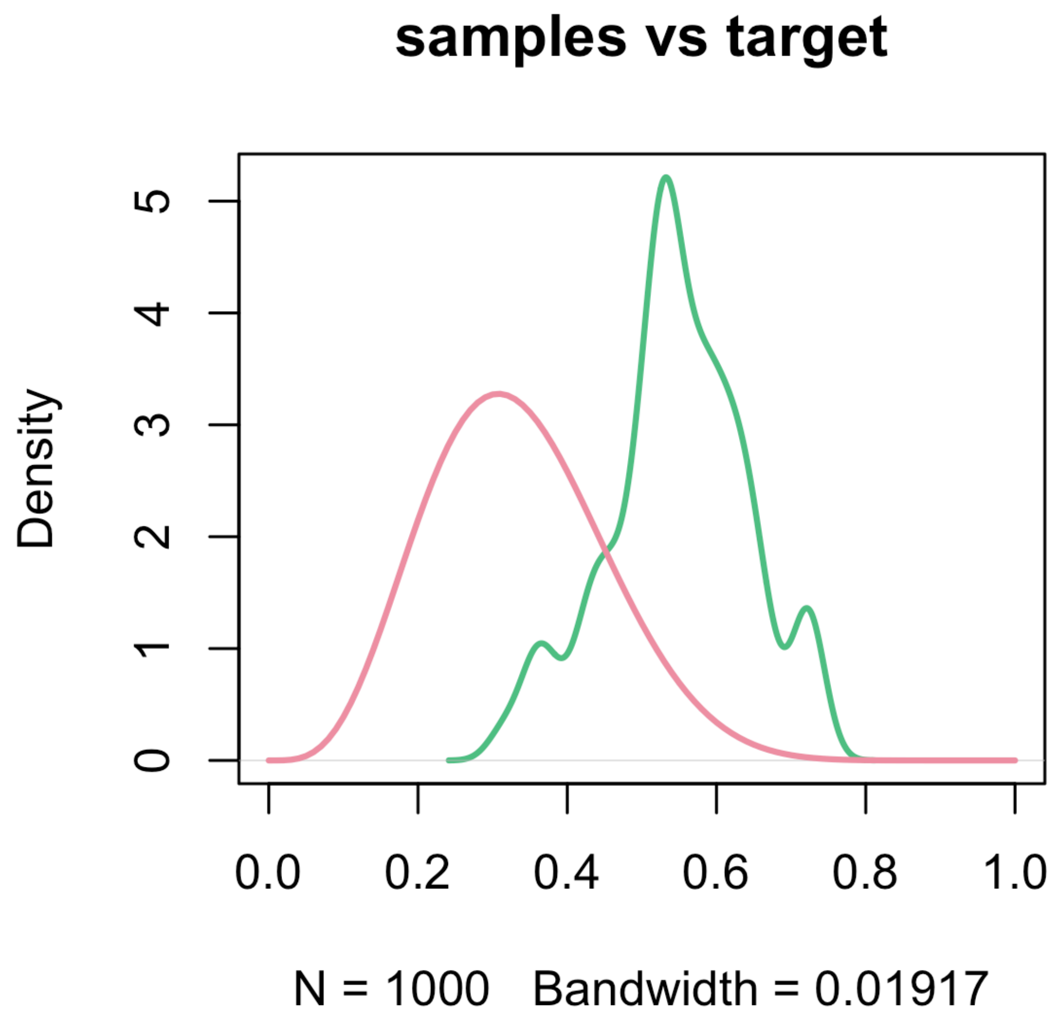
```
print(sprintf("Effective sample size: %.2f, Rejection Rate: %.2f"
              effectiveSize(rw_small), rejectionRate(as.mcmc(rw_s
```

```
## [1] "Effective sample size: 4.42, Rejection Rate: 0.05"
```

The Metropolis Algorithm

$\tau^2 = 0.01$ ("small" jump)

Ran MCMC for 1000 iters.



Terrible Error.

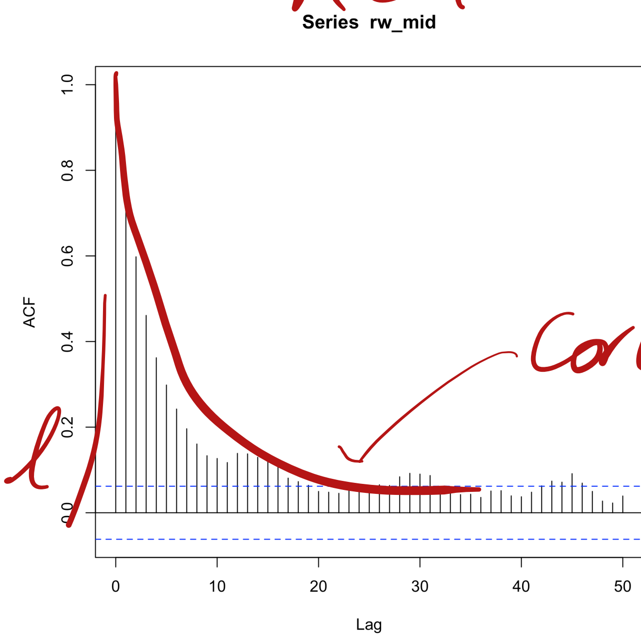
The Metropolis Algorithm

$\tau^2 = 0.1$ ("medium" proposal variance)

```
acf(rw_mid, lag=50)
```

ACF

$Cor(\theta_t, \theta_{t+l})$



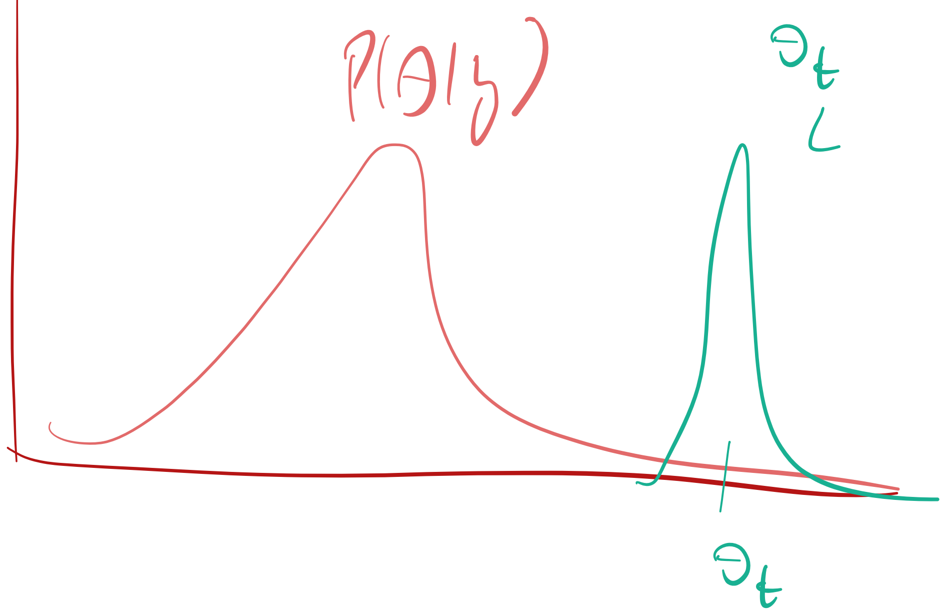
$Cor(\theta_t, \theta_{t+20}) \approx 0$

```
print(sprintf("Effective sample size: %.2f, Rejection Rate: %.2f"
              effectiveSize(rw_mid), rejectionRate(as.mcmc(rw_mid
```

[1] "Effective sample size: 128.86, Rejection Rate: 0.23"

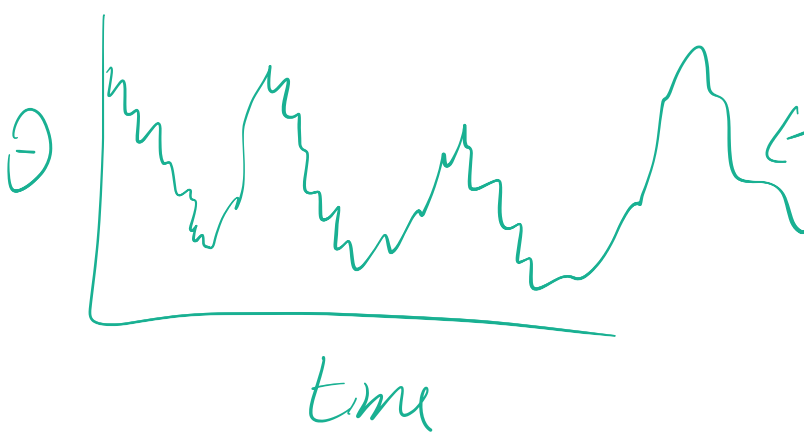
$J(\theta^* | \theta_t)$

$\sim N(\theta_t, \tau^2)$



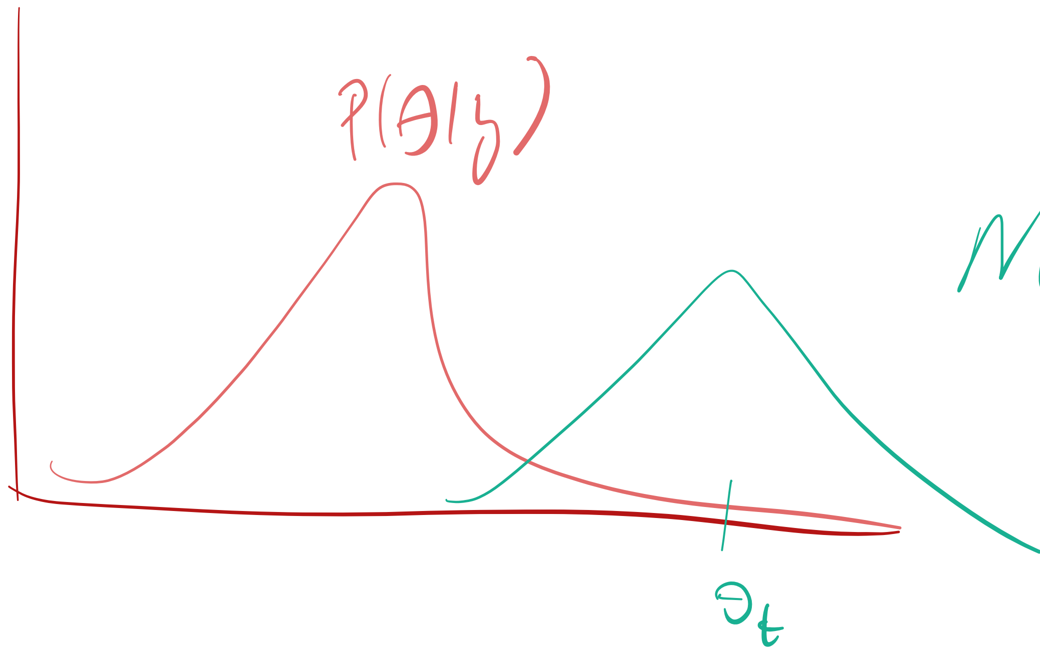
$\sim N(\theta_t, .01)$

Proposals are highly corelated
Can't move quickl.

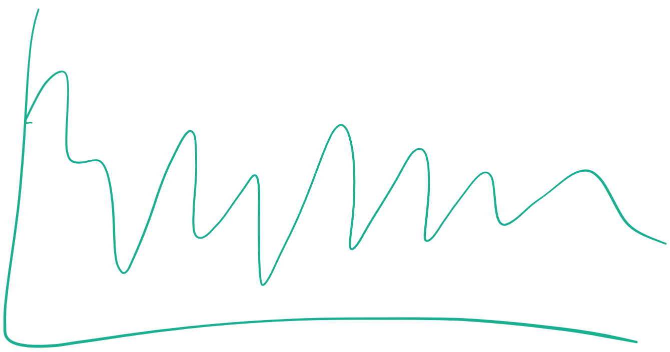


High autocorrelation

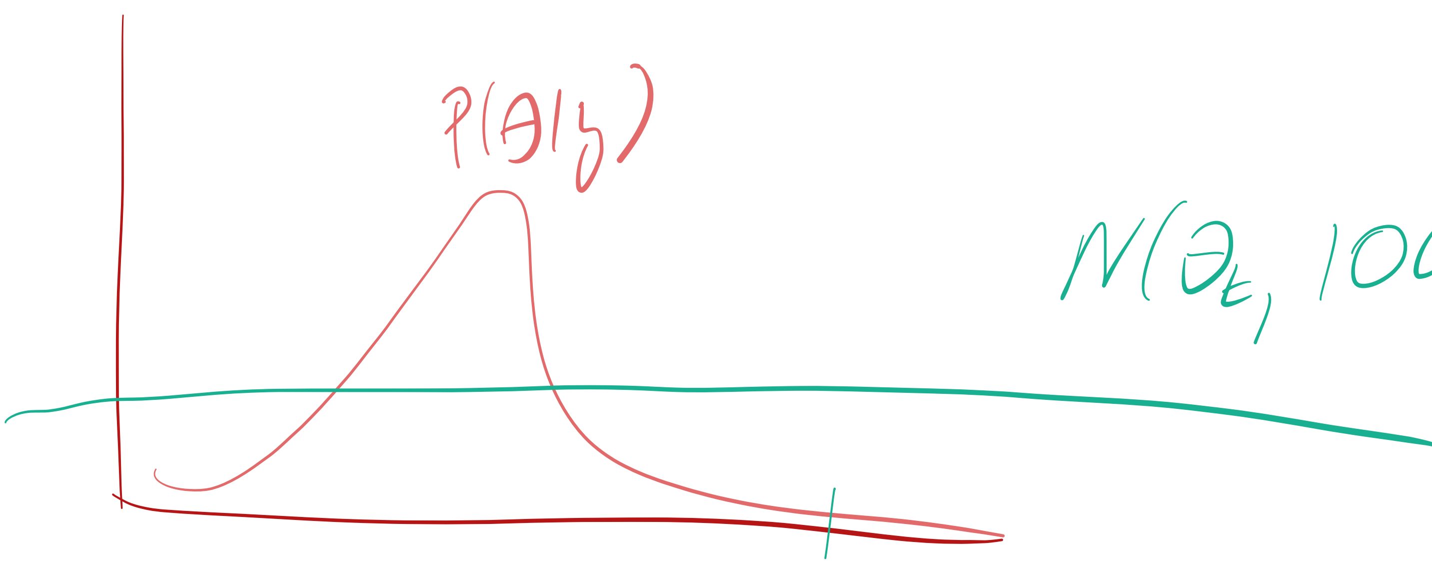
θ_{t+1} is "close" to θ_t



$N(\theta_t, .2)$



Less correlation.

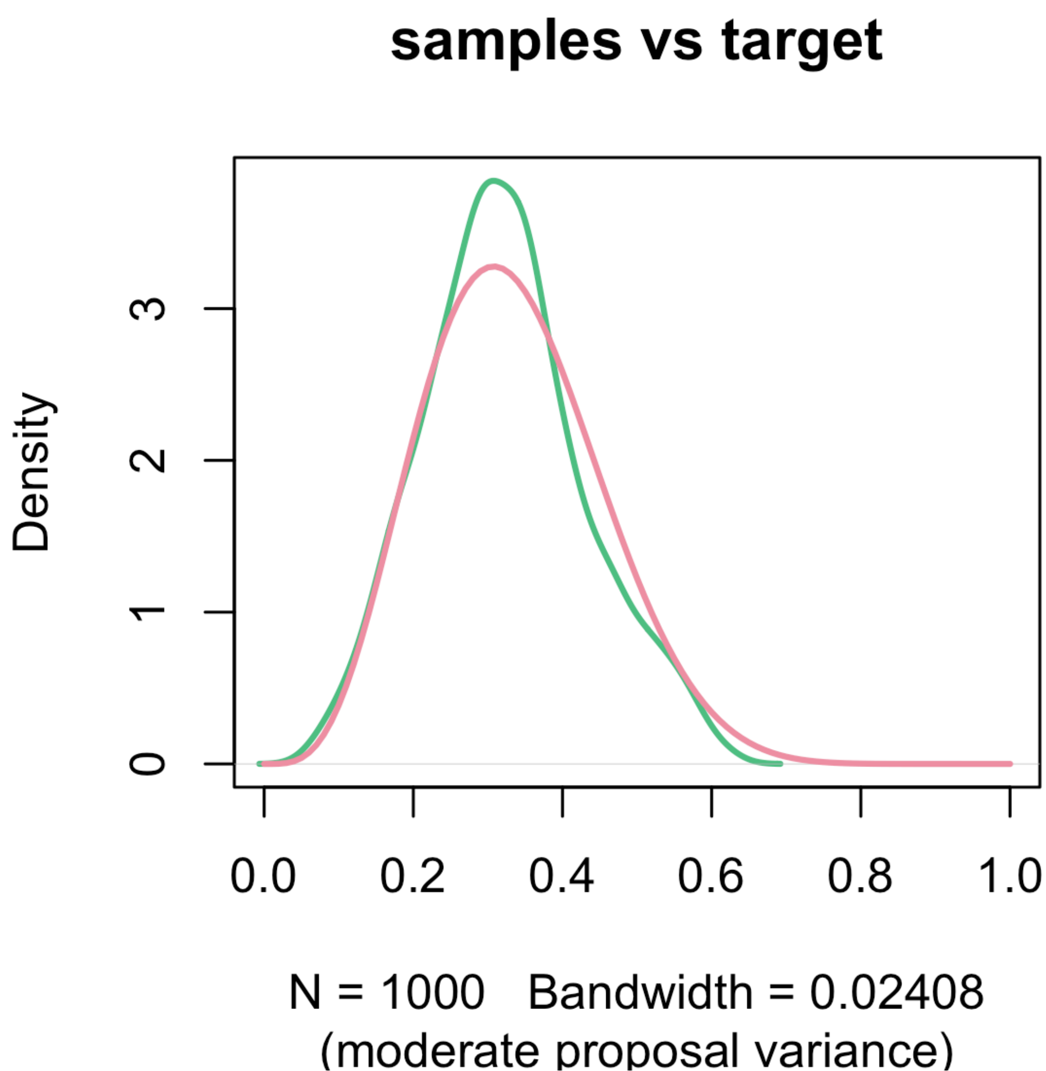


$N(\theta_t, 100)$

time

The Metropolis Algorithm

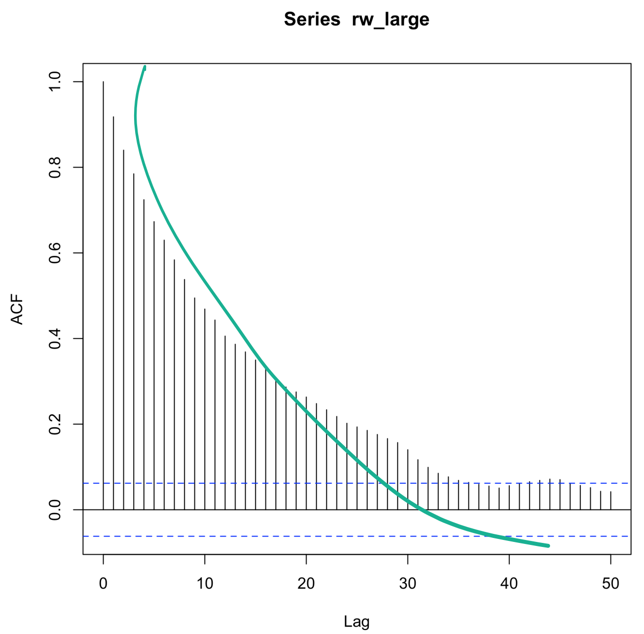
$\tau^2 = 0.1$ ("medium" proposal variance)



The Metropolis Algorithm

$\tau^2 = 2$ ("large" jump)

```
acf(rw_large, lag=50)
```

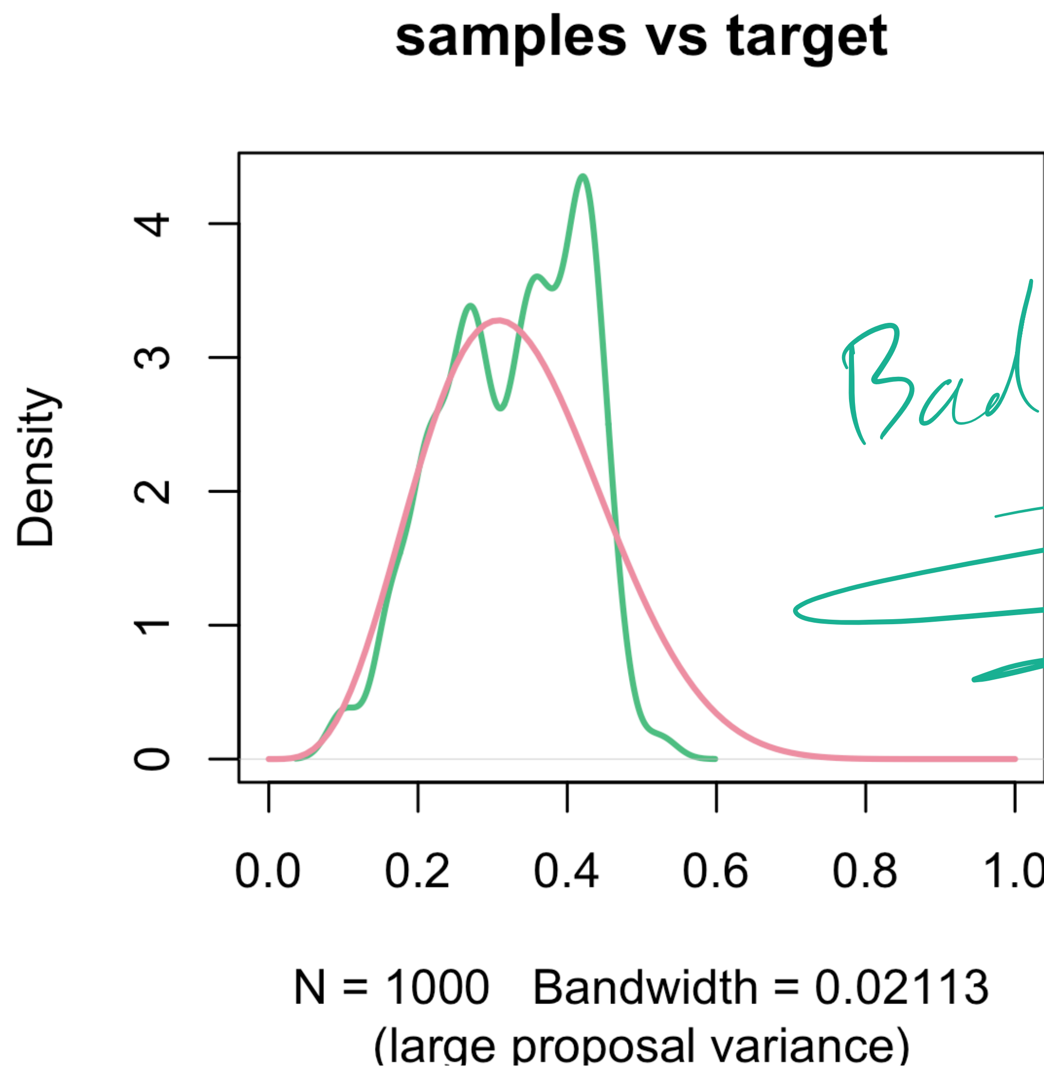



```
print(sprintf("Effective sample size: %.2f, Rejection Rate: %.2f"
              effectiveSize(rw_large), rejectionRate(as.mcmc(rw_la
```

```
## [1] "Effective sample size: 32.51, Rejection Rate: 0.93"
```

The Metropolis Algorithm

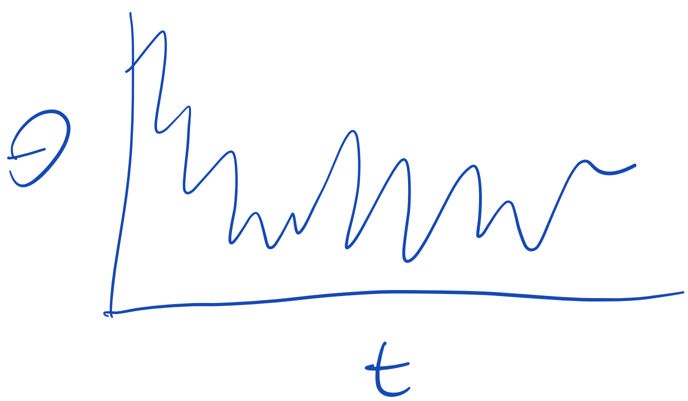
$\tau^2 = 2$ ("large" proposal variance)



Bad Fit

Trace Plot:

Plot of parameter(s) vs time (iteration)



Small, Moderate and Large Proposal variance

"City Scape" traceplot ⇒ High Rejection Rate

True Posterior

MCMC approx.

τ^2 big

$J(\theta^*|\theta_t) \sim N(\theta_t, \tau^2)$

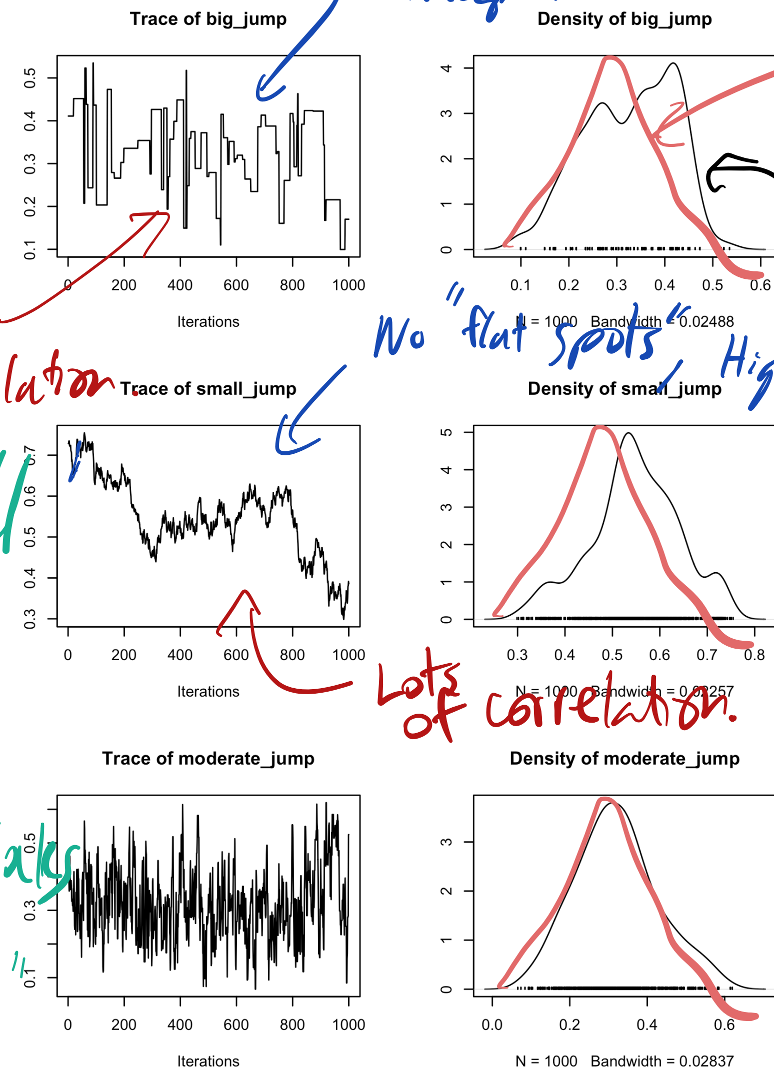
Lots correlation

No "flat spots", High Acceptance.

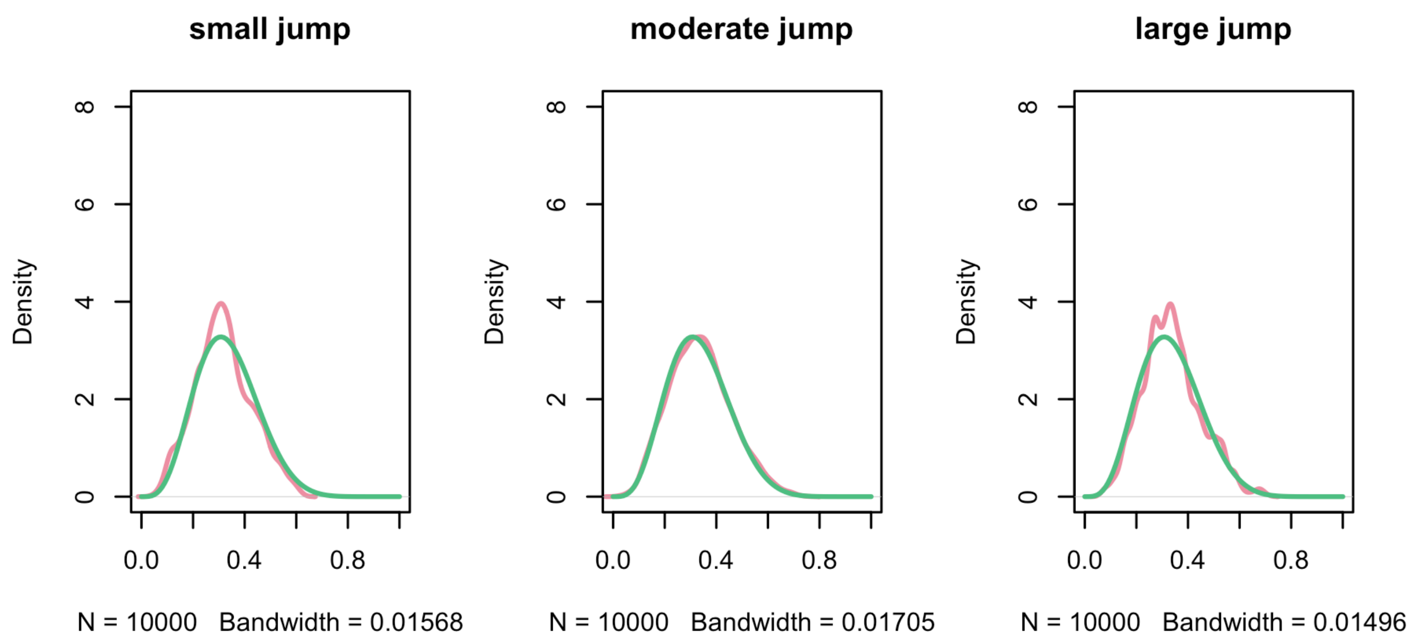
τ^2 small

Lots of correlation.

τ^2 "goldilocks zone"



10,000 Samples




```
## [1] "Effective sample size (small proposal variance): 20.68"
## [1] "Effective sample size (medium proposal variance): 1041.74"
## [1] "Effective sample size (large proposal variance): 418.34"
```

MCMC diagnostics

- Diagnose the chain performance by examining:
    - Rejection rate
    - Autocorrelation
    - Effective sample size
    - Trace Plot (visual inspection)

MCMC diagnostics

Too High

- **Rejection rate**: if rejection rate is high, the proposal density is proposing "too far away" from the current sample
- Means we are often keeping the previous sample
- Sampler is "sticky". Traceplots look like cityscapes.

Fraction of proposals that aren't used.

Too Low

- Accept too much, not exploring parameter space fast enough.

MCMC diagnostics

- **Autocorrelation**: if samples are highly correlated, the proposal density is proposing "too close" to the current sample
    - Highly correlated implies the Markov chain is mixing slowly

$\Rightarrow$ High mc error.

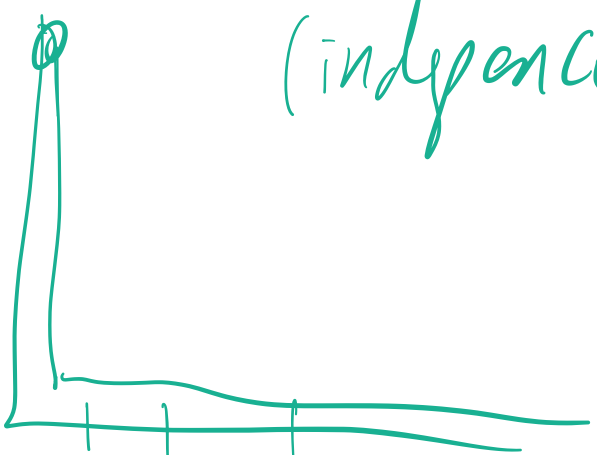
- The mixing time of a Markov chain is the time until the Markov chain is "close" to its limiting distribution.

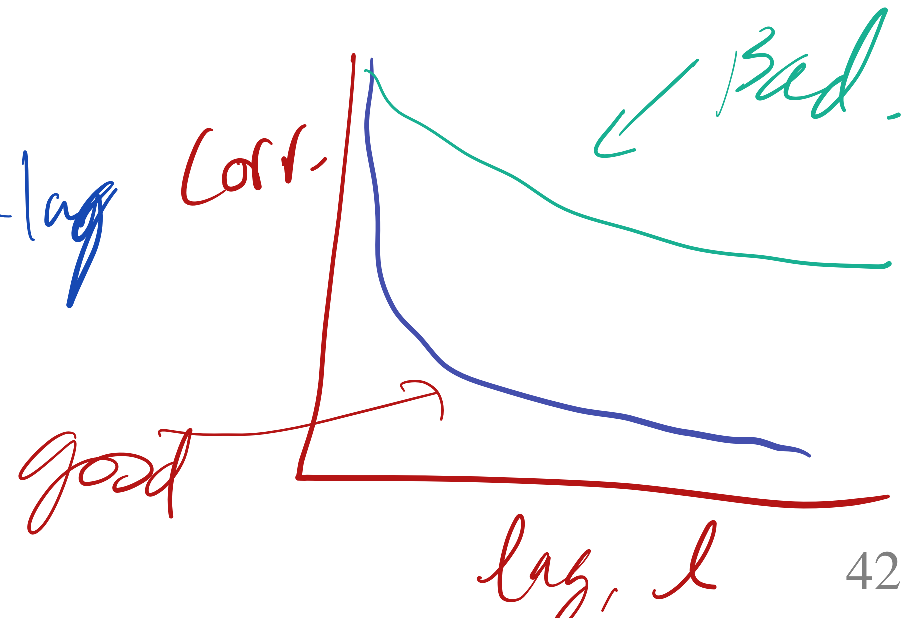
$$\mathrm{Cor}(\theta_t, \theta_{t+l})$$

lag

'acf' function

Acf (indepence)





MCMC diagnostics

- **Effective sample size:** correlated chain of samples is equivalent to this number of independent samples (in terms of MC error)
  - High rejection rate implies a lot of duplicate samples so effective size is smaller than number of iterations
  - High autocorrelation means neighboring samples are very similar (even if not exactly the same)

For indepent samples, MC error is proportional to $1/S_{ESS}$

For dep. samples: $= \dfrac{\sum\sum \mathrm{cor}(\theta_i, \theta_j | y)}{S^2}$

Solve For S_{ess}

Metropolis Algorithm

- In the Beta example, the accept ratio, $\min(1, \frac{p(\theta^*|y)}{p(\theta_t|y)})$ is zero when $\theta^* > 1$ or $\theta^* < 0$
- If τ^2 (proposal variance) too large, you will often reject your proposal
  - Proposing far from your current location may move you too far out of the high density areas
  - This makes for a "sticky" chain (stay at current sample for a long time)

"City Scape"

- τ^2 too small, the chain explore the parameter space slowly

high acf,

- A rule of thumb is to aim for 30%-40% acceptance rate for random walk samplers.
  - This balances "stickiness" and slow convergence

Metropolis-Hastings Algorithm

- The Metropolis-*Hastings* algorithm allows us to use non-symmetric proposals
- The Hastings correction is needed when $J(\theta^* \mid \theta_t) \neq J(\theta^t \mid \theta^*)$

$$r = \min(1, \frac{p(\theta^* \mid y)}{p(\theta_t \mid y)} \boxed{\frac{J(\theta^t \mid \theta^*)}{J(\theta^* \mid \theta_t)}})$$

Hastings correction.

- For symmetric proposals $\frac{J(\theta^t|\theta^*)}{J(\theta^*|\theta_t)} = 1$

MCMC for multivariate distributions

- Modeling wing length of different specifies of midge (small, two-winged flies)
- Reminder: $Y_i \sim N(\mu, \sigma^2)$
- $P(\mu \mid \sigma^2), \mu \sim N(\mu_0, \frac{\sigma^2}{\kappa_0})$
- $P(\sigma^2) \propto \frac{1}{\sigma^2}$ (improper prior)

MCMC for multivariate distributions

Example: midge wing length

- Modeling wing length of different specifies of midge (small, two-winged flies)

- From prior studies: mean wing length close to 1.9mm with sd close to 0.1mm

- $\mu_0 = 1.9, \sigma_0^2 = 0.01$

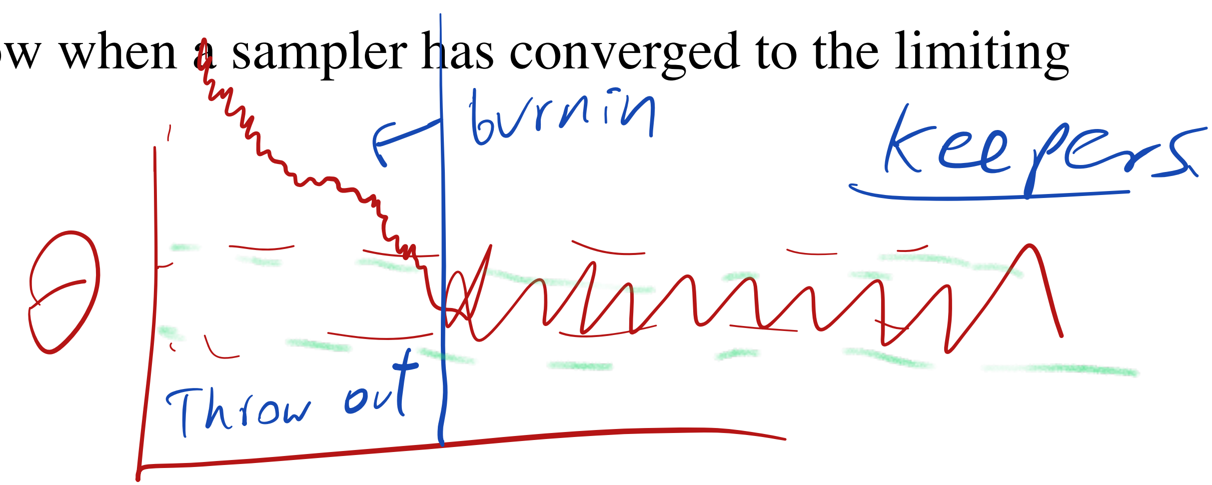
- Choose $\kappa_0 = 1$

- We will run 2 separate chains at different starting locations

- $J(\mu*, \log(\sigma*)) \sim N((\mu^t, \log(\sigma^t)), \begin{pmatrix} \tau_\mu^2 & 0 \\ 0 & \tau_{log\sigma}^2 \end{pmatrix})$

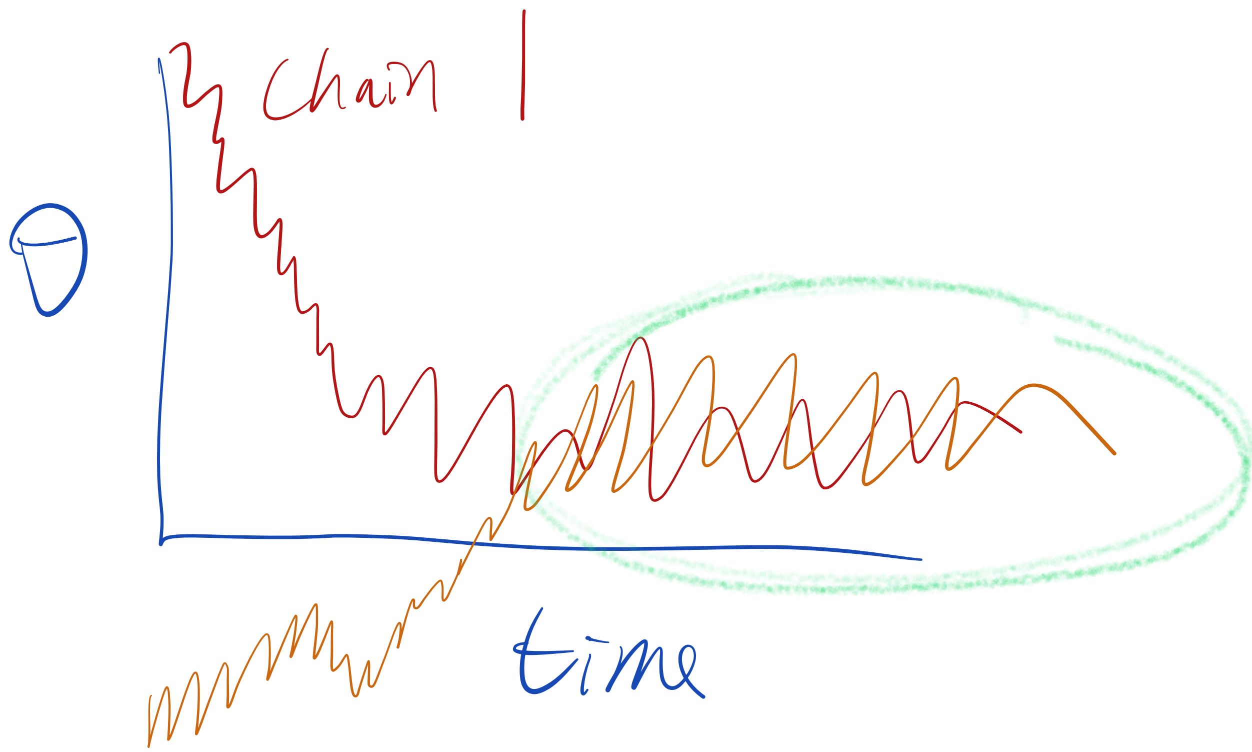
Initialization and Convergence

- In the long run, it doesn't matter where you initialize your sampler
- In practice, we can only run an algorithm for a finite amount of time
    - Need to check with the sampler has converged to the limiting distribution
    - Exclude samples close to the initial values since these are unlikely to be representative samples
    - We call the time to convergence **burn in** and throw away and samples generated during this time.
- How do we know when a sampler has converged to the limiting distribution?



Run multiple Chains in Parallel

(Different Initializations)



time

Running multiple chains

- How do we know when a sampler has converged to the limiting distribution?
    - Hard to know for sure.
- Idea: run multiple chains at very different initial locations
    - If the chains are very far apart we haven't converged
    - If the chains end up in "the same place", we have confidence that its reached convergence

Diagnosing mixing with Rhat

$B = \frac{n}{m-1} \sum_{j=1}^{m} \left(\bar{\psi}_{.j} - \bar{\psi}_{..}\right)^2$

$W = \frac{1}{m} \sum_{j=1}^{m} s_j^2$

Demo

Multiple Chains in Stan

```
library(cmdstanr)
y <- c(1.64, 1.7, 1.72, 1.74, 1.82, 1.82, 1.82, 1.9, 2.08)

# compile stan model.  This may take a minute.
stan_model <- cmdstan_model(stan_file="normal_model.stan")

## data is a list, arguments must match the datablock in the stan
stan_fit <- stan_model$sample(data=list(N=length(y), y=y, k0=1),
                              chains = 2,
                              num_warmup=200,
                              num_samples = 1000,
                              refresh=200)
```

Printing,

Burnin

keep.

2x 1000 total Samples.

Multiple Chains in Stan

```
## Running MCMC with 2 sequential chains...
##
## Chain 1 finished in 0.0 seconds.
## Chain 2 finished in 0.0 seconds.
##
## Both chains finished successfully.
## Mean chain execution time: 0.0 seconds.
## Total execution time: 0.3 seconds.
```

```
SAMPLING FOR MODEL 'normal_model' NOW (CHAIN 1).
Chain 1:
Chain 1: Gradient evaluation took 9e-06 seconds
Chain 1: 1000 transitions using 10 leapfrog steps per transition would take 0.09 seconds.
Chain 1: Adjust your expectations accordingly!
Chain 1:
Chain 1:
Chain 1: Iteration:   1 / 1000 [  0%]  (Warmup)
Chain 1: Iteration: 200 / 1000 [ 20%]  (Warmup)
Chain 1: Iteration: 201 / 1000 [ 20%]  (Sampling)
Chain 1: Iteration: 400 / 1000 [ 40%]  (Sampling)
Chain 1: Iteration: 600 / 1000 [ 60%]  (Sampling)
Chain 1: Iteration: 800 / 1000 [ 80%]  (Sampling)
Chain 1: Iteration: 1000 / 1000 [100%]  (Sampling)
Chain 1:
Chain 1:  Elapsed Time: 0.015009 seconds (Warm-up)
Chain 1:                0.02576 seconds (Sampling)
Chain 1:                0.040769 seconds (Total)
Chain 1:

SAMPLING FOR MODEL 'normal_model' NOW (CHAIN 2).
Chain 2:
Chain 2: Gradient evaluation took 1.5e-05 seconds
Chain 2: 1000 transitions using 10 leapfrog steps per transition would take 0.15 seconds.
Chain 2: Adjust your expectations accordingly!
Chain 2:
Chain 2:
Chain 2: Iteration:   1 / 1000 [  0%]  (Warmup)
Chain 2: Iteration: 200 / 1000 [ 20%]  (Warmup)
Chain 2: Iteration: 201 / 1000 [ 20%]  (Sampling)
Chain 2: Iteration: 400 / 1000 [ 40%]  (Sampling)
Chain 2: Iteration: 600 / 1000 [ 60%]  (Sampling)
```

Stan Samples

```r
draws <- stan_fit$draws(format="df")
draws
```

μ σ

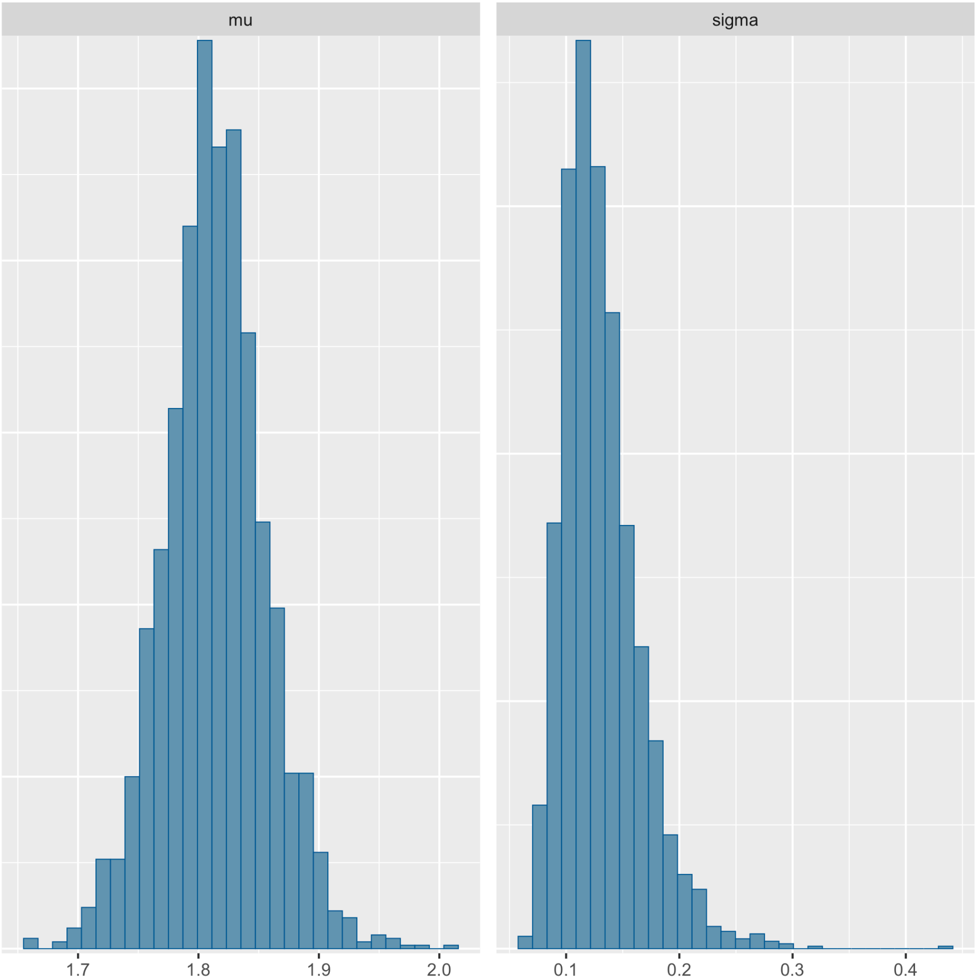
log P(θ|y)

```
## # A draws_df: 1000 iterations, 2 chains, and 3 variables
##     lp__  mu sigma
## 1     17 1.9  0.16
## 2     17 1.8  0.15
## 3     18 1.8  0.14
## 4     17 1.8  0.16
## 5     18 1.8  0.11
## 6     18 1.8  0.12
## 7     18 1.8  0.12
## 8     18 1.8  0.11
## 9     16 1.8  0.20
## 10    17 1.9  0.15
## # ... with 1990 more draws
## # ... hidden reserved variables {'.chain', '.iteration', '.draw'}
```

Plotting Stan Draws

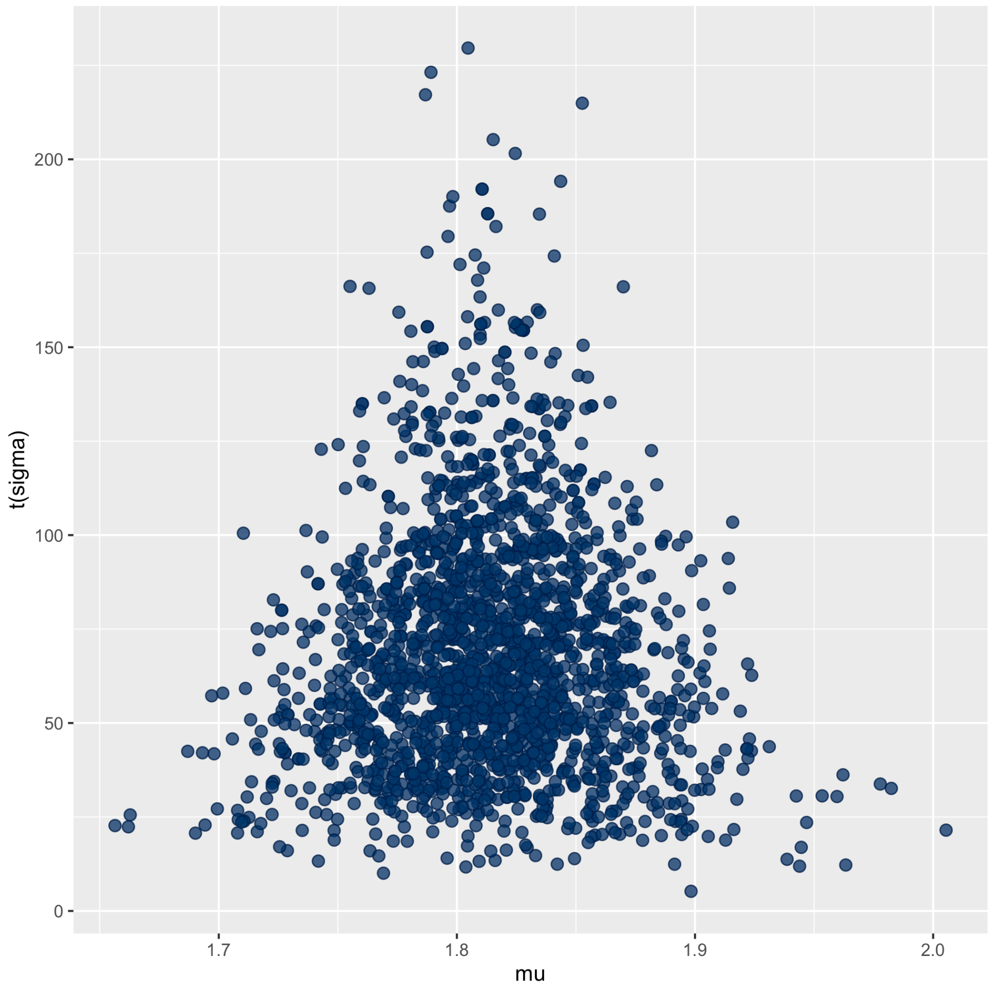
```
mcmc_hist(draws, pars=c("mu", "sigma"))
```

```
## `stat_bin()` using `bins = 30`. Pick better value with `binwidth`.
```



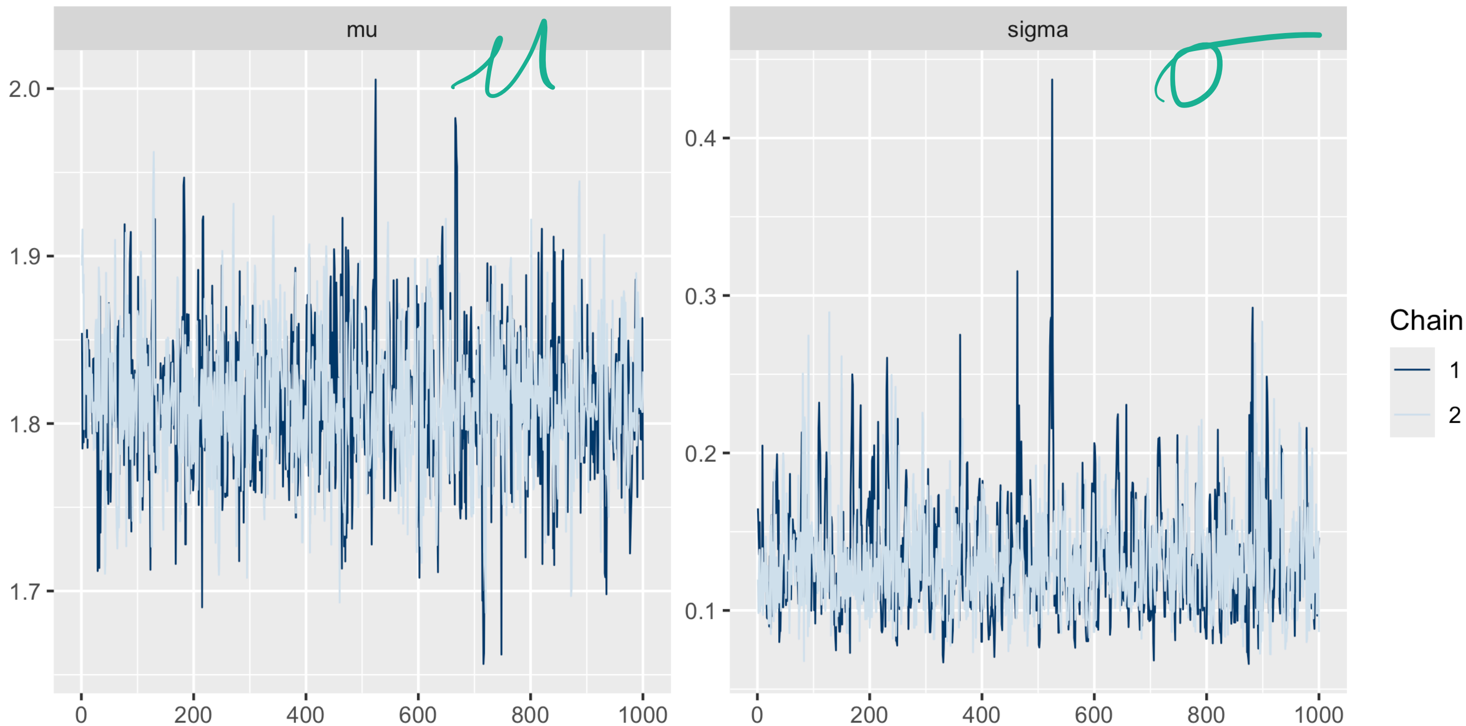
Plotting Stan Draws

```
mcmc_scatter(draws,
             pars=c("mu", "sigma"),
             transformations=list(sigma=function(x) 1/x^2))
```



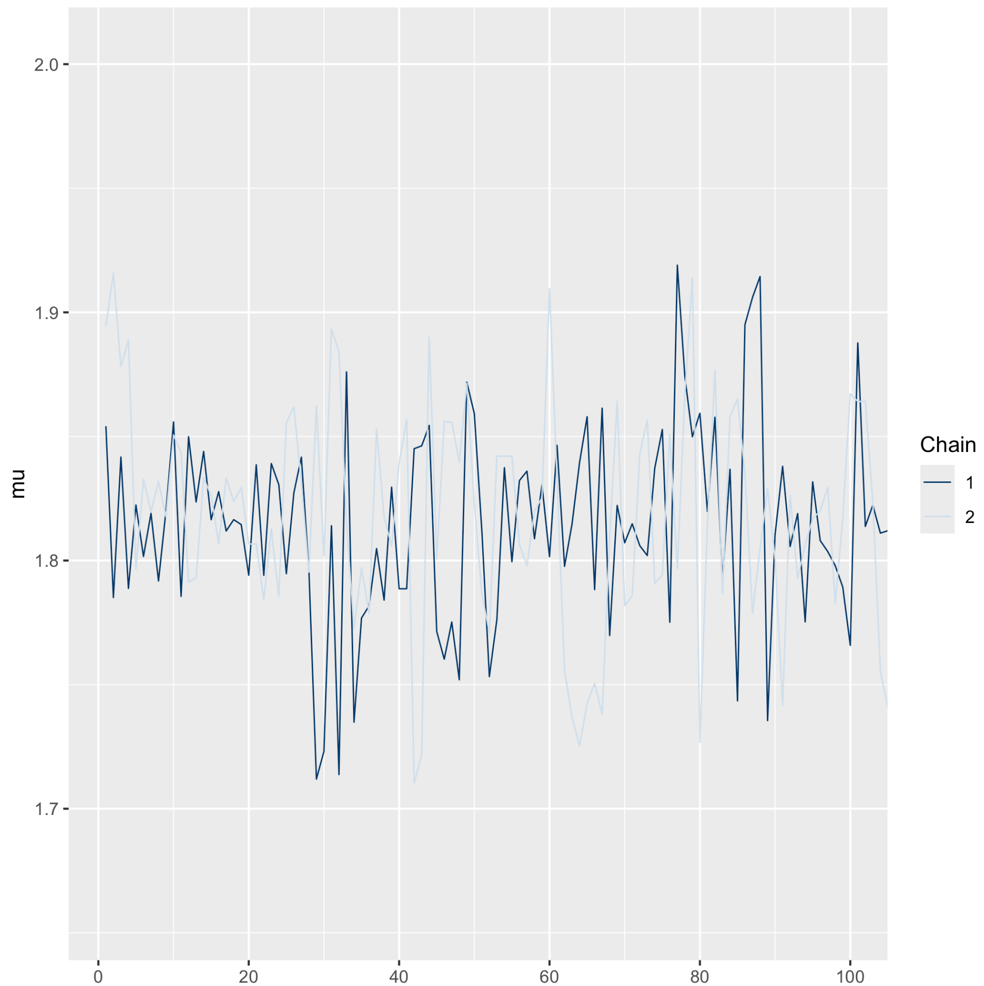
MCMC Diagnostics: Traceplots

```
mcmc_trace(draws, pars=c("mu", "sigma"))
```



MCMC Diagnostics: Traceplots

```
mcmc_trace(draws, pars=c("mu"), window = c(1, 100))
```



MCMC Diagnostics

Efficiency can be expressed as (effective samples) / (total iterations)

```
stan_fit$summary() %>% glimpse
```

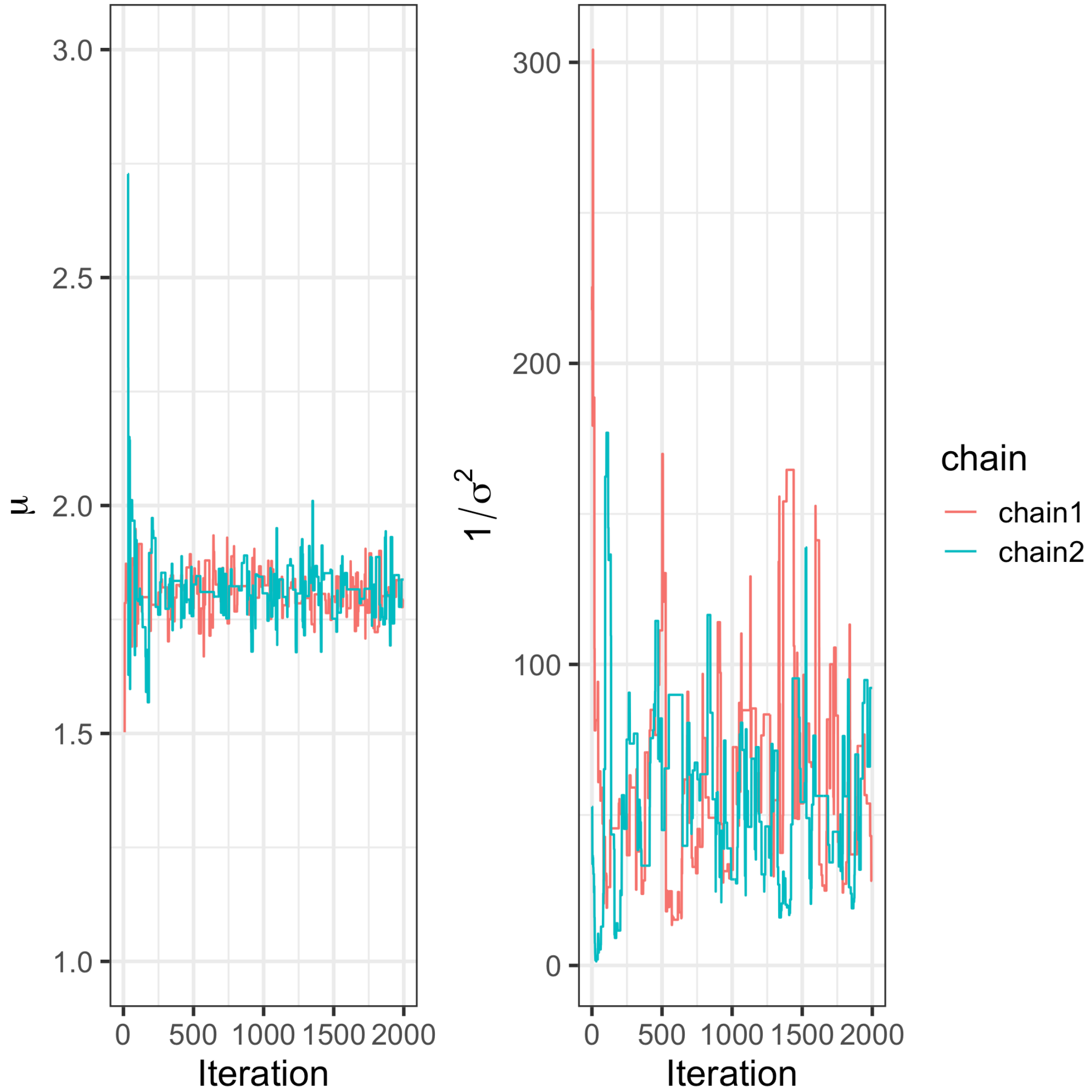
```
## Rows: 3
## Columns: 10
## $ variable <chr> "lp__", "mu", "sigma"
## $ mean     <dbl> 17.2356446, 1.8138022, 0.1308553
## $ median   <dbl> 17.5798500, 1.8129550, 0.1251045
## $ sd       <dbl> 1.12399902, 0.04227812, 0.03454343
## $ mad      <dbl> 0.82136040, 0.03759132, 0.02915681
## $ q5       <dbl> 15.01483500, 1.74625850, 0.08693568
## $ q95      <dbl> 18.3126400, 1.8858515, 0.1942028
## $ rhat     <dbl> 1.000395, 1.000797, 1.000443
## $ ess_bulk <dbl> 843.4363, 1394.9833, 778.1124
## $ ess_tail <dbl> 937.1279, 1085.1102, 757.5916
```

Post. mean

Post. Quantiles

Effective S.S.

MCMC for multivariate distributions



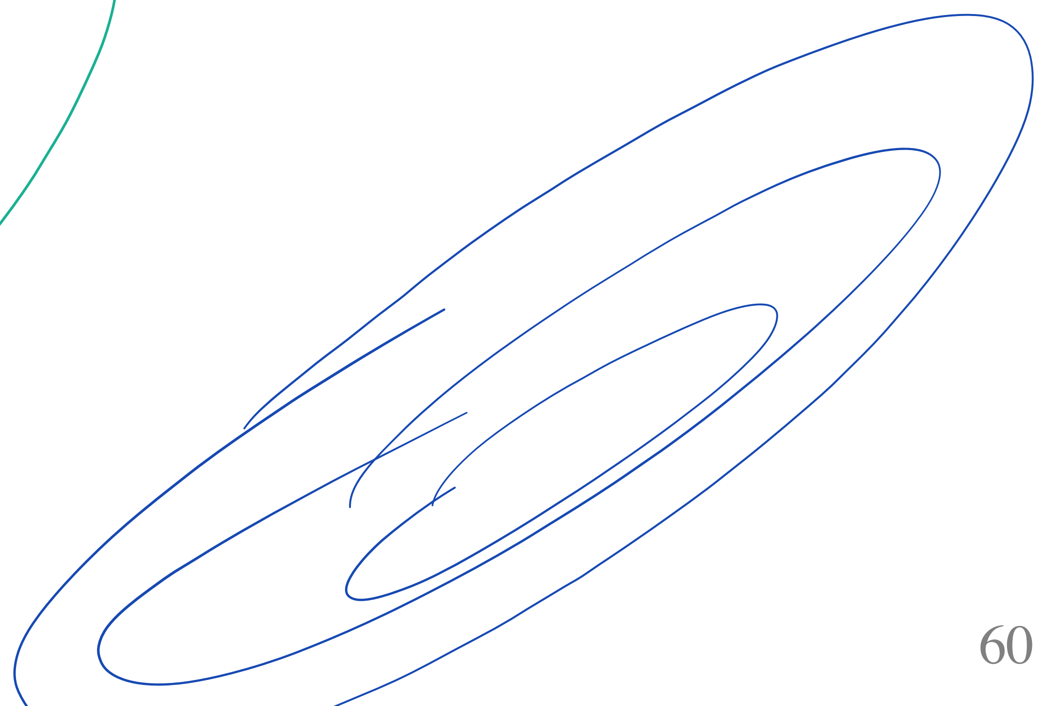
MCMC for multivariate distributions

- We will run 2 separate chains at different starting locations
- For chain 1: $J(\theta^* \mid \theta_t) = N\left(\begin{pmatrix} \mu^{(t)} \\ \log(\sigma^{(t)}) \end{pmatrix}, \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}\right)$

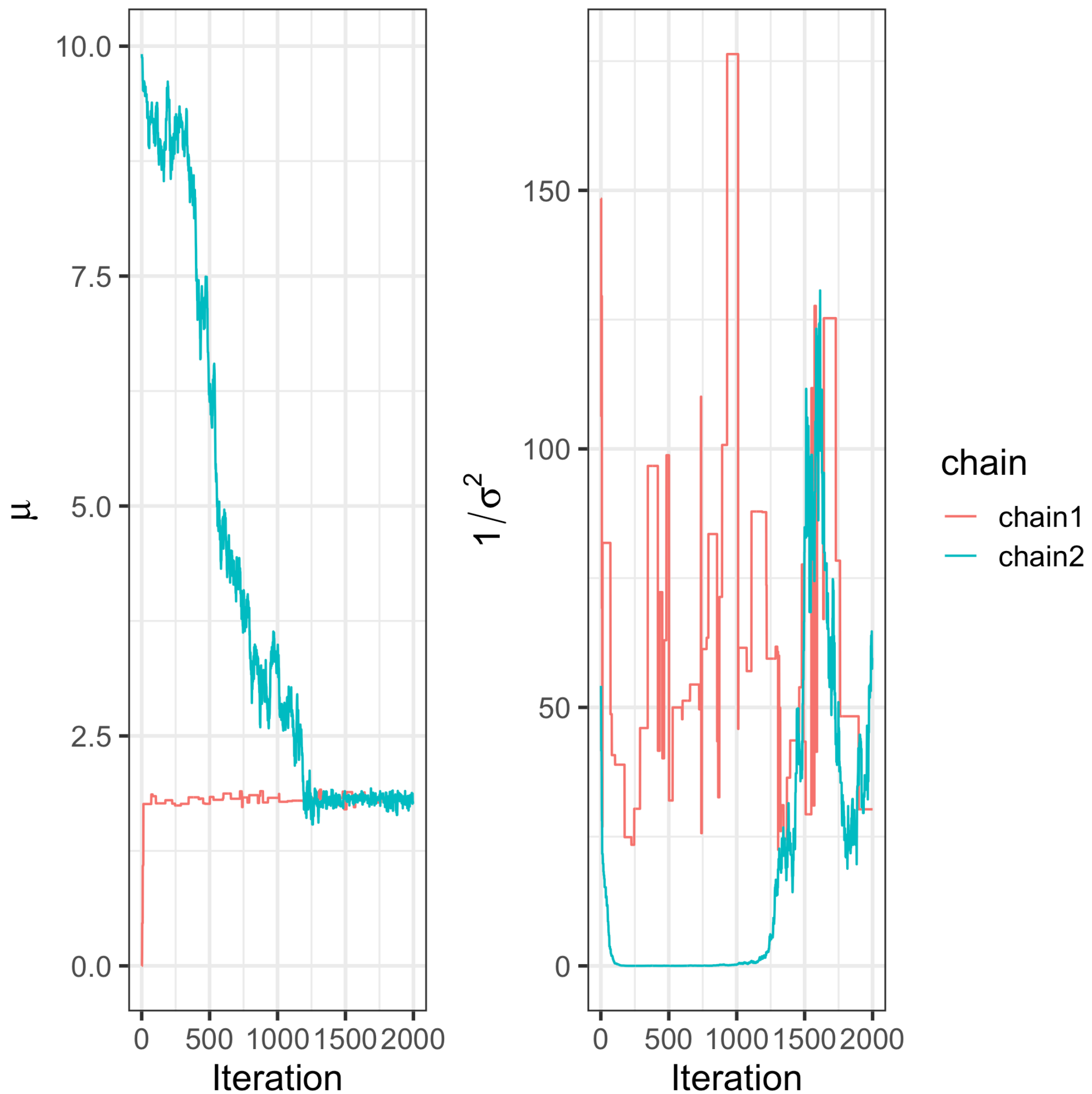
How to choose?

- For chain 2: $J(\theta^* \mid \theta_t) = N\left(\begin{pmatrix} \mu^{(t)} \\ \log(\sigma^{(t)}) \end{pmatrix}, \begin{pmatrix} 0.1 & 0 \\ 0 & 0.1 \end{pmatrix}\right)$

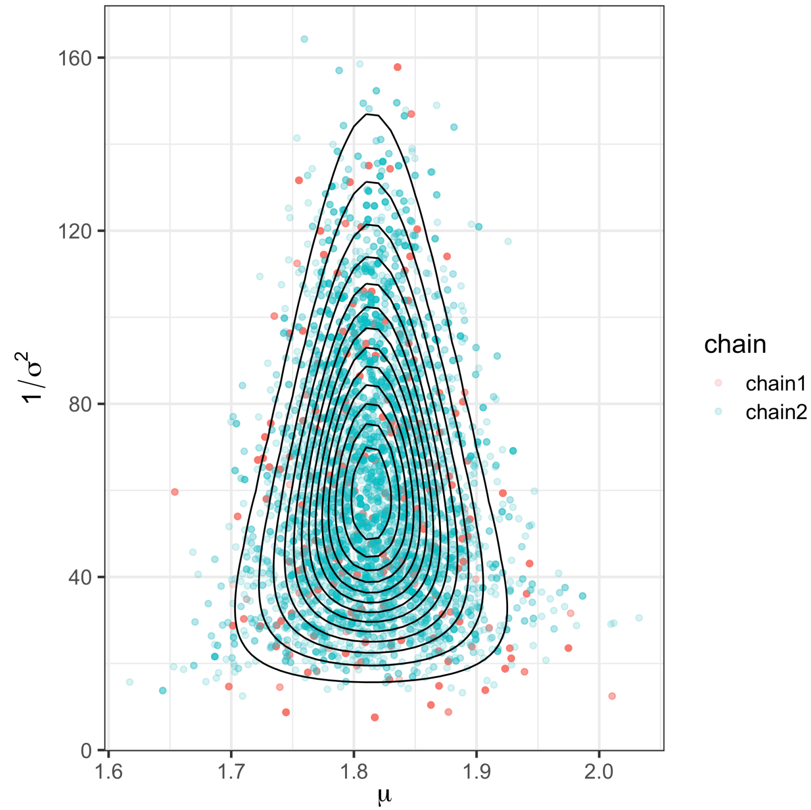
(μ_t, σ_t)



MCMC for multivariate distributions



MCMC for multivariate distributions




```
## # A tibble: 2 × 3
##   chain   EF_mu EF_prec
##   <fct>   <dbl>   <dbl>
## 1 chain1 389.     108.
## 2 chain2   7.43    35.6
```

Metropolis Sampling

Try out the Metropolis algorithm at:

https://chi-feng.github.io/mcmc-demo/app.html

- Choose "Random Walk MH" algorithm
- Experiment by sampling from different target distribution
- Try different proposal variances by changing "Proposal σ"

Gibbs Sampling

The Gibbs Sampler

- The Gibbs sampler is actually special case of the MH sampler
    - This is not obvious or immediately apparent
- Idea: break the problem down into many smaller sampling problems
- Iteratively update each parameter in the full parameter vector by doing getting lower-dimensional sample

The Gibbs Sampler

- Suppose that the parameter vector θ can be divided into d subvectors.
- For iterations, $s = 1, \ldots, S$:
  - For $j \in 1, \ldots, d$
  - Draw a value from the conditional distribution of θ_j given all the other parameters,

$$\theta_j^s \sim p(\theta_j | \theta_{-j}^{s-1}, y),$$

where θ_{-j}^{t-1} has consists of updated parameters for all parameters preceding j and the previous iteration's values for all succeeding parameters,

$$\theta_{-j}^{t-1} = \left(\theta_1^t, \ldots, \theta_{j-1}^t, \theta_{j-1}^{t-1}, \ldots, \theta_d^{t-1}\right).$$

The Gibbs Sampler

- To identify the full conditional distributions:

  1. Write down the full posterior
  2. For each parameter, θ_j, remove all multiplicative constants that don't have a θ_j in them.
  3. Identify the type of distribution to sample from

- Assuming conjugate prior distributions, the full conditionals in the normal model are:

  - $p(\mu \mid \sigma^2, y)$ is a normal distribution
  - $p(\frac{1}{\sigma^2} \mid \mu, y)$ is a gama distribution

The Gibbs Sampler

Gibbs Sampling a Bivariate Normal

Demo

Gibbs Sampler

- Advantages
  - proposals, $p(\theta^{(t+1)} \mid \theta^{(t)})$, are never rejected.
  - don't need to choose the proposal density distribution or tune parameters of the density
- Disadvantages
  - It can be difficult to derive the full-conditional distributions (unless, for example, the prior distributions are chosen to be conjugate)
  - When the parameters of the posterior distribution are highly correlated:
  - Hard to traverse diagonals and consequently:
    - autocorrelation of the samples is high
    - effective sample size is low

Challenges in MCMC

- Modern models often have *many* parameters. Large models pose a challenge for MCMC.
- When there are thousands or more parameters
  - MCMC may take a long time to conververge to the limiting distribution
  - In Metropolis-Hastings we have many tuning parameters for the proposal distribution
  - Gibbs sampling has no tuning parameters, but does not work well for highly correlated posterior distributions
- In general, MCMC is slow relative to optimization methods

Summary of MCMC

What is Monte Carlo and MCMC:

- MCMC is *not* a model
- It does *not* generate more information
- Provides *dependendent* approximate samples from $p(\theta \mid y)$
- Samples can be used to summarise $p(\theta \mid y)$ (approximate integrals)

Computational Considerations

- For very small values of $p(\theta \mid y)$, numerical underflow is a problem
- Can resolve this by working on the log scale

```
dnorm(1000) / dnorm(1001)
```

```
## [1] NaN
```

```
dnorm(1000, log=TRUE) - dnorm(1001, log=TRUE)
```

```
## [1] 1000.5
```

- Compute $l = \min(0, \log(p(\theta^* \mid y)) - \log(p(\theta_t \mid y)))$

- If $\log(u) < l$ we accept θ^* as our next point

MCMC for multivariate distributions

- Modeling wing length of different specifies of midge (small, two-winged flies)
- Reminder: $Y_i \sim N(\mu, \sigma^2)$
- $P(\mu \mid \sigma^2), \mu \sim N(\mu_0, \frac{\sigma^2}{\kappa_0})$
- $P(\sigma^2), \sigma^2 \sim \text{Inv-Gamma}(\nu_0/2, \nu_0/2\sigma_0^2)$
- Parameterize in terms of $\theta = (\mu, \log(\sigma))$? Why parameterize this way?
- Let $J(\theta^* \mid \theta_t) = N\left(\begin{pmatrix} \mu^{(t)} \\ \log(\sigma^{(t)}) \end{pmatrix}, \begin{pmatrix} 0.5 & 0 \\ 0 & 0.5 \end{pmatrix}\right)$

Challenges in MCMC

- Modern models often have *many* parameters. Large models pose a challenge for MCMC.
- When there are thousands or more parameters
  - MCMC may take a long time to conververge to the stationary distribution
  - In Metropolis-Hastings we have many tuning parameters for the proposal distribution
  - Gibbs sampling has no tuning parameters, but does not work well for highly correlated posterior distributions
- In general, MCMC is very slow relative to optimization methods

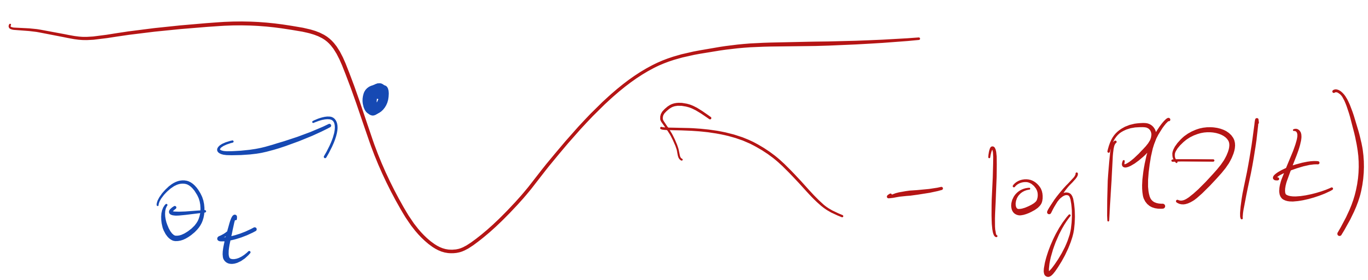
Modern MCMC

- Gibbs and Metropolis samplers have a "random walk" behavior
  - Induces autocorrelation
  - Makes it difficult to explore the posterior space
- Hamiltonian Monte Carlo (HMC) borrows an idea from physics to reduce this problem

HMC (Hamiltonian Monte Carlo)

- Imagine a marble on a frictionless surface. The location of the marble is the current value of θ_t
- The negative posterior density is the "height" of the surface
- Each iteration we flick the marble with some velocity in a random direction
- Regions of high posterior density are like "wells"

Stan uses it.



HMC

- For Metropolis-Hastings we only need to be able to evaluate the posterior at each location

$P(\theta | y)$

- For HMC we need the gradient (derivative) of the posterior as well
  - Determines where the marble rolls

$$\frac{d\,P(\theta|y)}{d\theta}$$

- In physics the Hamiltonian is the sum of the kinetic energies, plus the potential energy of the particles
  - As our proposal, we randomly sample a momentum for the marble and update its position accordingly
  - Can think of HMC as the MH algorithm with a very clever jumping/proposal rule

$J(\theta^* | \theta_t)$ s.t.

θ^* far from θ_t and

we accept.

HMC

Try out HMC at:

https://chi-feng.github.io/mcmc-demo/app.html

- Choose "HamiltonianMC" algorithm
- Experiment by sampling from different target distributions
- Compare to the Random Walk Metropolis

Approximate Inference

- MCMC can be very slow in high dimensional problems
- Idea: find a distribution that is easy to sample from which closely approximate $p(\theta \mid y)$
- A couple of examples
  - Laplace Approximation
  - Variational Bayes

Laplace Approximation to the Posterior

- Approximate the posterior distribution using a multivariate normal distribution
- When we have a lot of i.i.d. observations, the posterior will be approximately normal
- Center the normal at the mode of the posterior
- Compute the (co)variance of the normal by computing the second derivative / hessian of the posterior at the mode

Laplace Approximation

- Approximate the posterior distribution using a normal distribution
- When we have a lot of i.i.d. observations, the posterior will be approximately normal
- Center the normal at the mode of the posterior
- Compute the (co)variance of the normal by computing the second derivative / hessian of the posterior at the mode

Laplace Approximation

- Let $\tilde{\theta}$ be the mode of the of the posterior distribution
- Use a Taylor Series approximation the log-posterior around the mode is
  - $logP(\theta \mid y) \approx logP(\tilde{\theta} \mid y) - 1/2(\theta - \tilde{\theta})H(\theta - \tilde{\theta})$
  - $H = \frac{d^2}{d\theta^2} logp(\theta \mid y)$
  - Note, linear term falls out because derivative at the mode is zero
- $p(\theta \mid y) \approx N(\tilde{\theta}, I(\theta)^{-1})$

Finding the mode of the posterior distribution

- Calculus
  - Take the log
  - Differentiate, set to zero and solve
- Computational
  - `optim` in R for one dimensional posteriors
  - `optimise` in R for multivariate p

Variational Bayes

- Find even better approximations
- Let θ be d dimensional parameter vector
- Let $\epsilon \sim MVN_d(0, \Sigma)$
- Let g_λ be a class of flexible functions parameterized by λ
  - Neural networks!
- Solve an optimization problem:

$$\arg\min_\lambda \text{dist}(p(\theta \mid y)), p(g_\lambda(\epsilon)))$$

  - Minimize the "distance" between the true posterior and the approximate one.
- Sample ϵ from a multivariate normal. $g_\lambda(\epsilon)$ will be a sample from something close to $p(\theta \mid y)$